



Arquitectura de Computadores:

Tema 1: ***Entrada/Salida***

Objetivos

- Comprender los problemas que surgen al intercambiar información con el mundo exterior a través de los dispositivos periféricos.
- Conocer los mecanismos que proporcionan los computadores para
 - realizar ese intercambio y
 - sincronizarse con los periféricos
(p.e. mecanismo de interrupciones)
 - Transferir datos
(p.e. DMA)

Bibliografía recomendada

- Stallings, W. “Organización y arquitectura de computadores”. Prentice Hall. 7ª Edición. 2006.
- Patterson, D. A., Hennessy, J. L. “Computer Organization and Design”. Morgan-Kaufmann. 4ª edición. 2009.
- de Miguel, P. “Fundamentos de los computadores”. Paraninfo. 9ª edición. 2004

Índice

- Introducción a E/S y módulos de E/S
 - Instrucciones de E/S
 - Tipos
 - Direcccionamiento
 - Ciclos de bus
 - Técnicas de E/S
 - E/S programada
 - E/S por interrupciones
 - E/S por DMA

Índice

- Entrada/Salida programada
 - Introducción
 - Ejemplo: módulo de E/S + *driver*
- Entrada/salida por interrupciones
 - Interrupciones
 - Interrupciones, excepciones y subrutinas
 - Solicitud de interrupciones
 - Servicio a interrupciones
 - Modo supervisor
 - Instrucciones para habilitar e inhibir el servicio de interrupciones

Índice

- Interrupciones (cont.)
 - Secuencia de reconocimiento de interrupciones
 - Rutina de servicio de interrupciones
 - Salvaguarda del estado del programa
 - Actualización del estado interno de la rutina
- Ejemplo de entrada/salida por interrupciones
- Operación con múltiples periféricos
 - Operación por muestreo (*polling*)
 - Identificación
 - Prioridades
 - Anidamiento

Índice

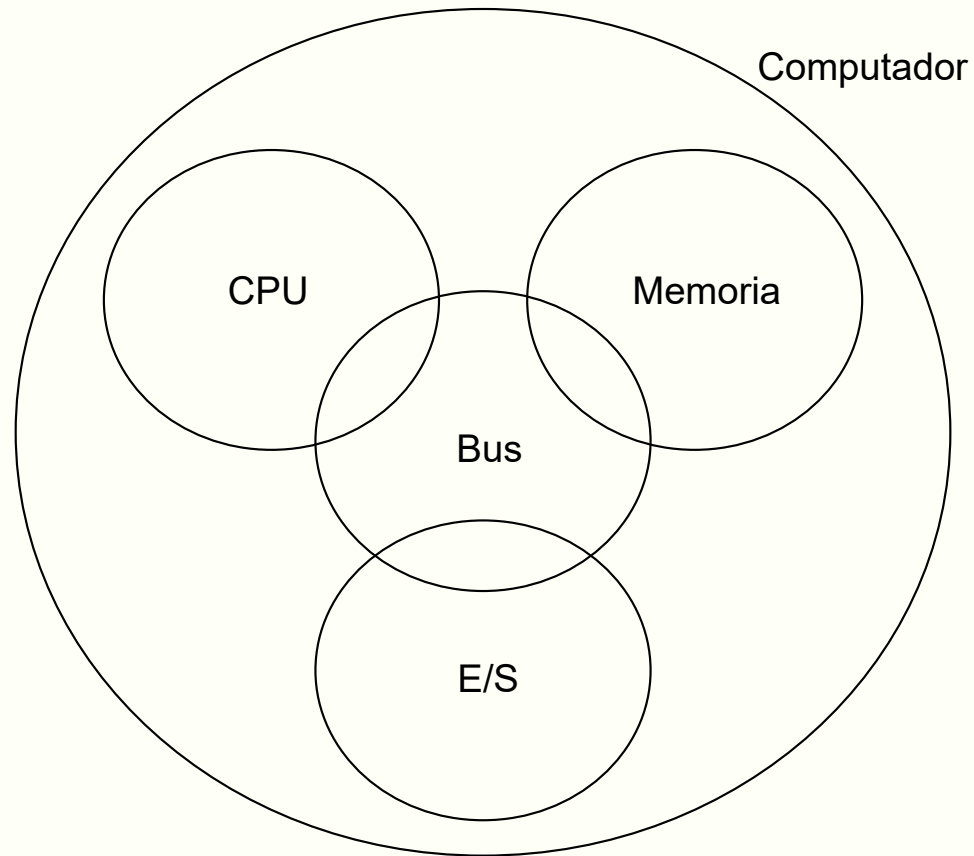
- Operación con múltiples periféricos (cont.)
 - Operación mediante vectorización
 - Módulo de entrada/salida con vectorización
 - Secuencia de reconocimiento de interrupciones
 - Esquema de prioridades hardware
 - Gestión centralizada
 - Gestión encadenada o *daisy chain*
 - Anidamiento de rutinas de servicio
 - Ejemplo
- Asignación de prioridades de interrupción
 - Interrupciones no enmascarables
- Conclusiones

Índice

- Entrada/salida mediante acceso directo a memoria (*DMA*)
 - Robo de ciclo aislado
 - Robo de ciclo en ráfagas
 - Módulo de entrada/salida con *DMA*

Periféricos

■ Introducción



Periféricos

- Ejemplos:
 - “Domésticos”:
 - Ratón
 - Teclado
 - Impresora
 - Disco duro (HDD/SDD)
 - Controlador LAN / WLAN
 - Etc., etc.
 - “Industriales”
 - Sensor de temperatura
 - Motores, p.e. orientación telescopio, posición robot, ...
 - Sistema de control de actitud de un satélite artificial
 - Etc., etc.

Problemática de la E/S

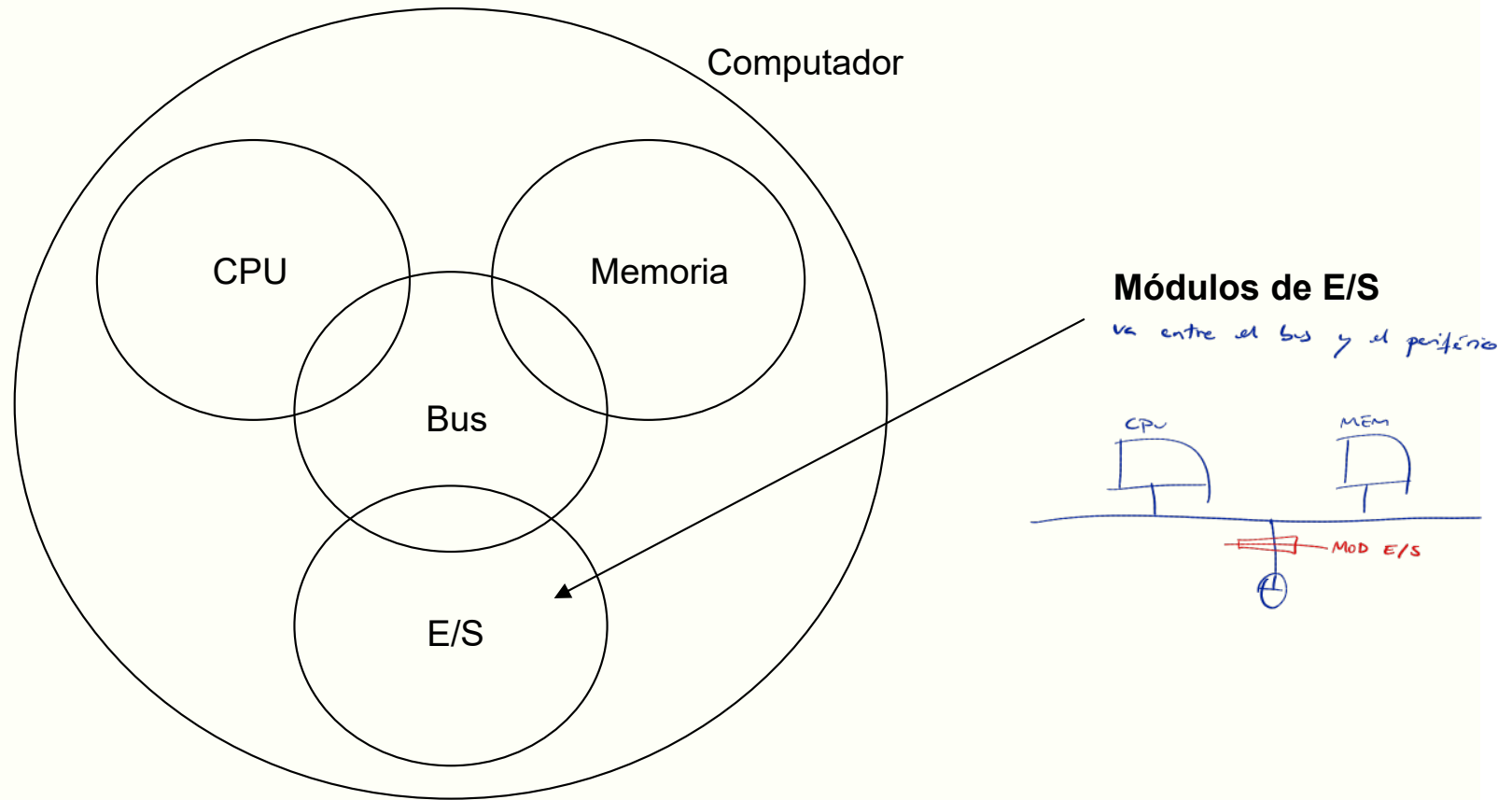
- Gran diversidad de periféricos con características muy diferentes :
 - Modo de funcionamiento (E, S, E/S, distintas órdenes,..)
 - Formato y tamaño de los datos
 - Velocidad de transferencia
 - Tiempo de acceso

→ es necesario “unificar” la visión Hw de los periféricos
hardware

→ **Módulos de E/S:** ocultan las particularidades de cada periférico. La CPU sólo dialoga con los módulos, todos iguales (estándar)

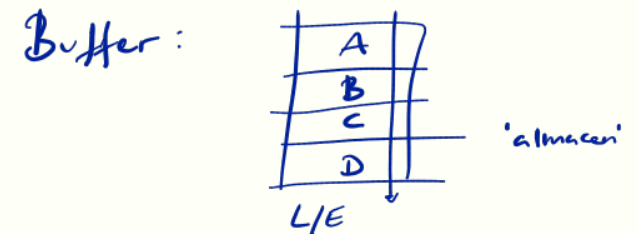
(Ignorar los detalles de cada periférico y tratar a todos igual.)

Módulos de E/S

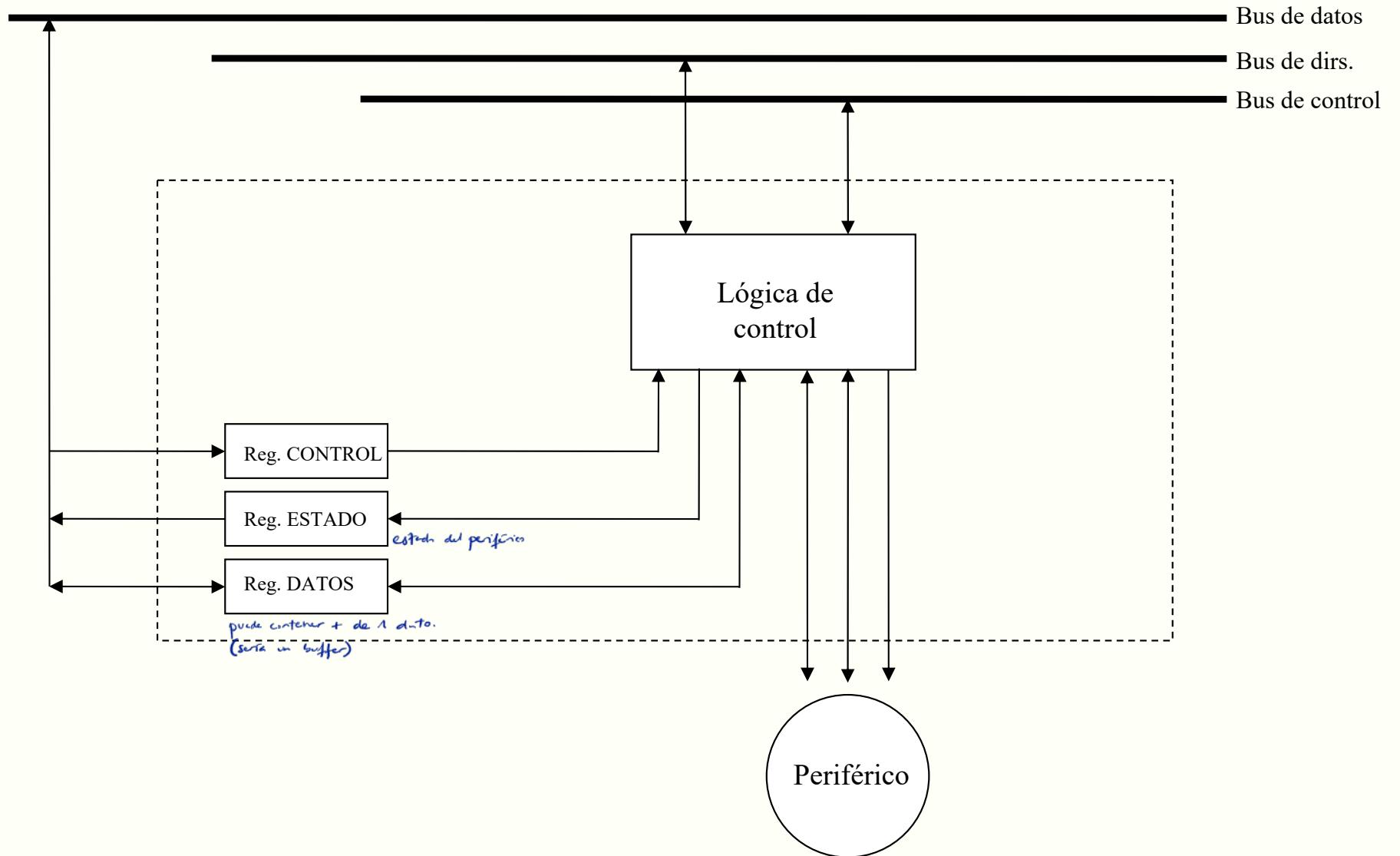


Módulos de E/S

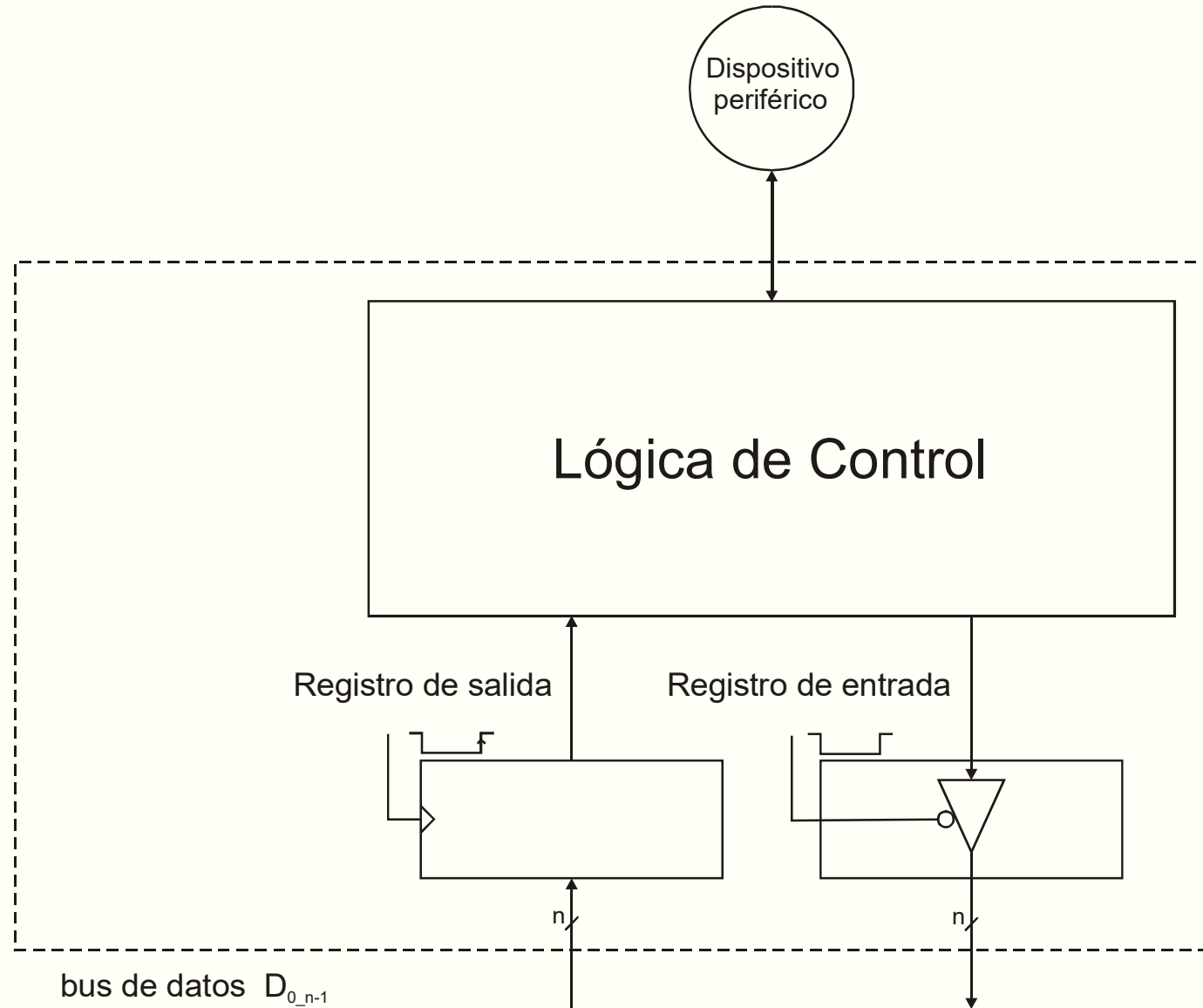
- Tienen dos interfaces, uno para dialogar con la CPU (estándar) y otro con el periférico (propio)
- Sus funciones son:
 - Control y temporización de la comunicación
 - Comunicación con la CPU
 - Comunicación con el periférico
 - *Buffering*, por la diferencia de velocidades
 - Control de errores



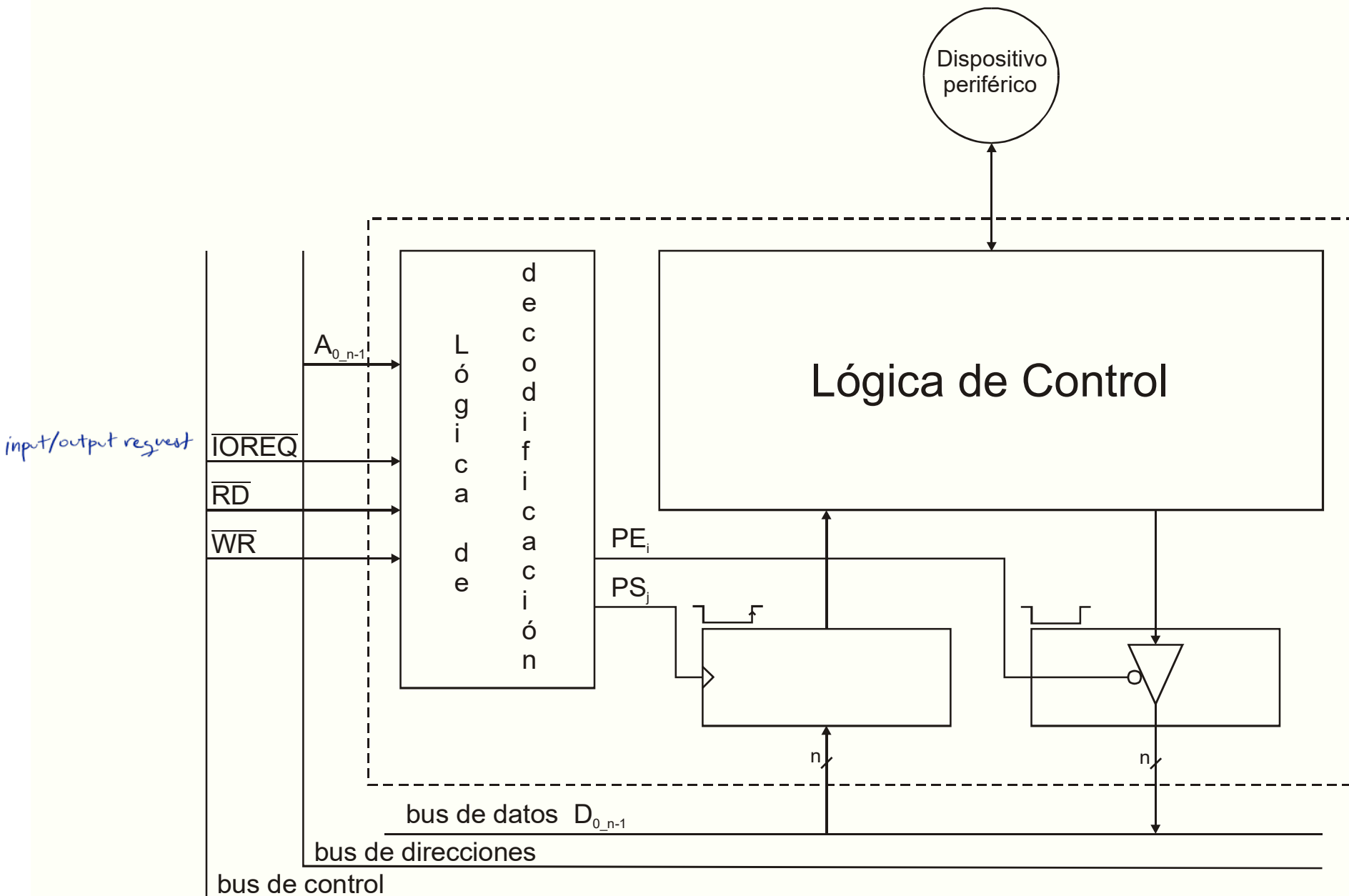
Módulos de E/S



Módulos de E/S

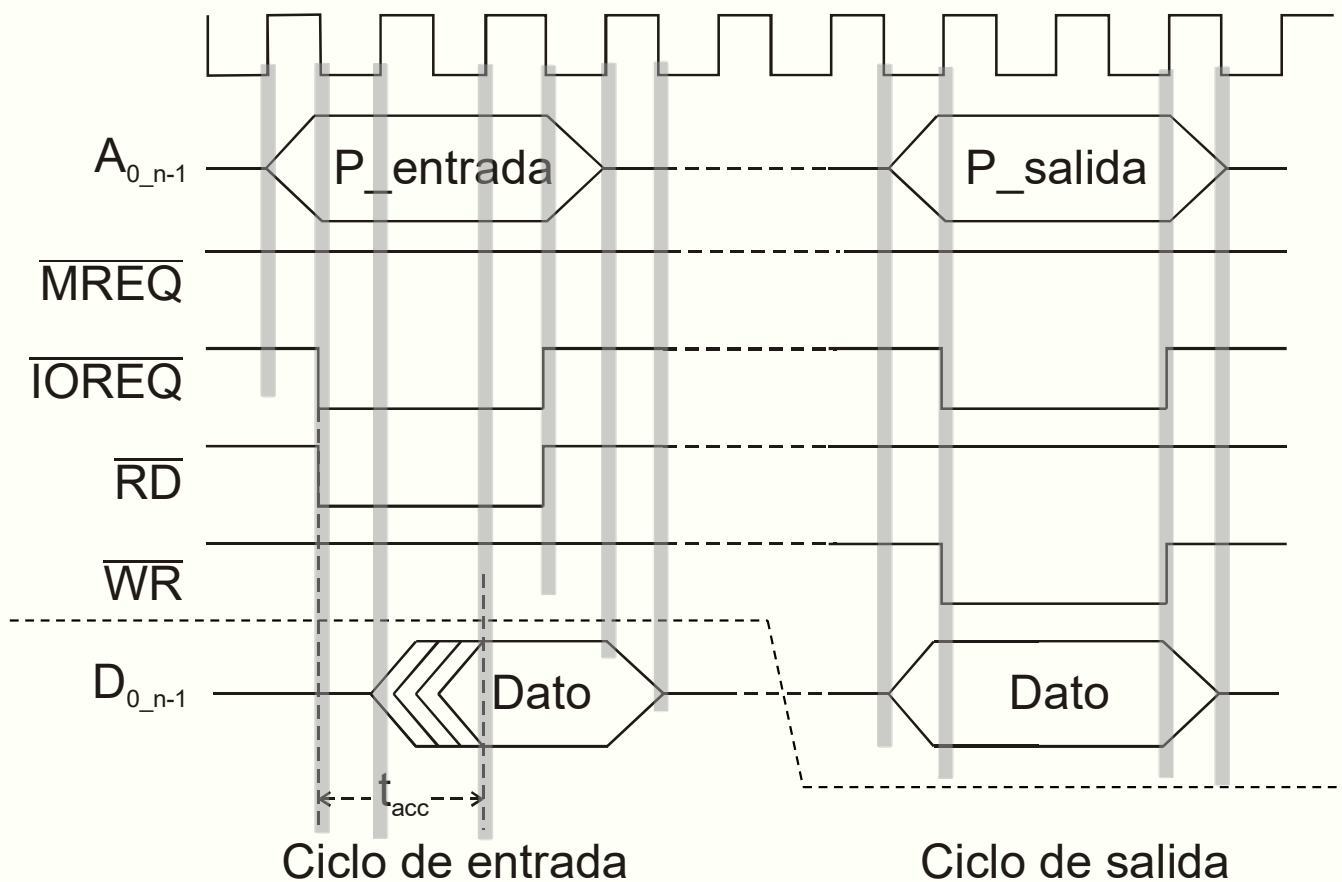


Módulos de E/S



Ciclos de bus de E/S

- Operación básica para intercambiar información entre elementos conectados al bus
- Ejemplos: Rd MEM, Wr MEM, Rd E/S, Wr E/S, Rd Bloque, RMW, etc.
- Secuencia de eventos (p.e. Wr): Dir $\rightarrow$ BusDir; Dato $\rightarrow$ BusDatos, 0 $\rightarrow$ IOREQ ...



Instrucciones de E/S

- Instrucciones de la Arquitectura del computador para la transferencia entre los regs. de la CPU y los regs. de los Módulos de E/S:

OUT `.R1, /Dir_Reg_Cont_Px; (orig → dest)`

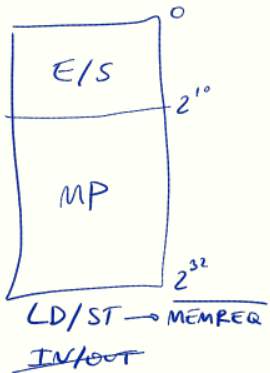
IN `.R1, /Dir_Reg_Dat_Px; (dest ← orig)`

- Algunas arquitecturas no usan instrucciones específicas, usan LD y ST

Direcciones de E/S

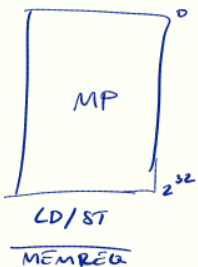
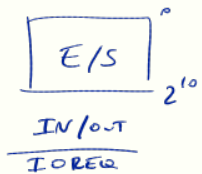
MP—memoria princ.

- *Mapa de direcciones único:* mismo ciclo de bus

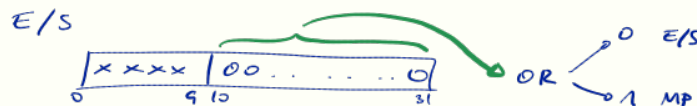


- Única línea de control MEMRQ (Memory Request) o AS (Address Strobe).
- Las instrucciones de memoria y de E/S son **ld** y **st** y se distingue el tipo por la dirección empleada.

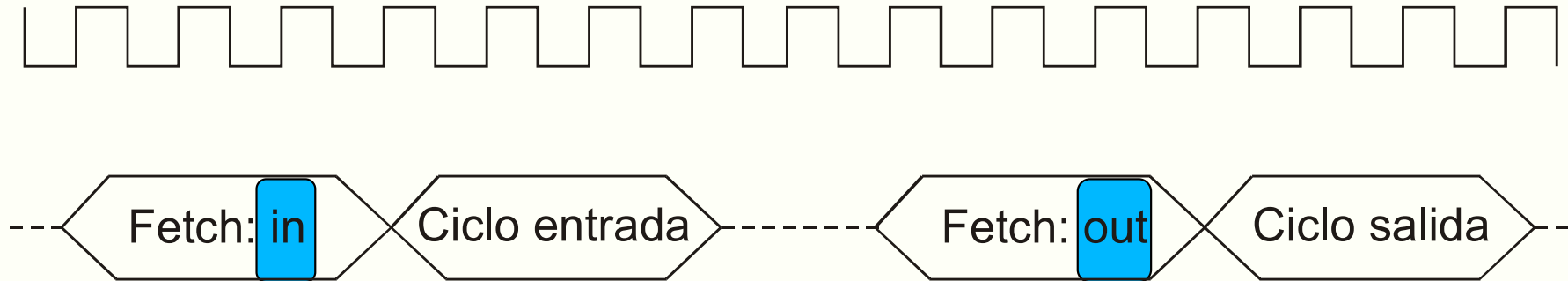
- *Mapas separados:* nuevo ciclo de bus para acceder a direcciones I/O



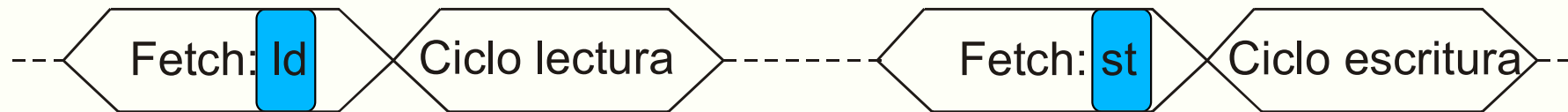
- Dos líneas de control: MEMRQ e IORQ (Input/Output Request).
- Las instrucciones específicas de E/S son **in** y **out** y se distinguen por su código de operación.



Direcciones de E/S



Mapas separados: Instrucciones de E/S



Mapa de direcciones único: E/S Mapeada en memoria

Decodificación de direcciones de E/S

- Direccionamiento **geográfico**: conjunto de direcciones prefijadas para cada ranura (slot)
 - *Demasiado rígido para módulos de E/S*
- Direccionamiento **lógico**: rango de direcciones configurable mediante interruptores en cada ranura
 - *Laborioso y propenso a errores*
- Ejemplo. Bus PCI (Plug and Play):
 - *Direccionamiento lógico configurable mediante registros de configuración.*
 - *Direccionamiento geográfico a los registros de configuración durante la iniciación.*
 - *Rutina inicial de configuración.*



Configuración de direcciones de E/S mediante interruptores



*Laborioso por no recordar donde conectaste cada cosa
y tener que buscarlo antes de poder modificar nada*

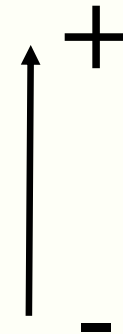
Técnicas de E/S

- Buscan un mejor aprovechamiento de la CPU
- La razón es que los periféricos son *MUY LENTOS*
- CPU, módulo E/S y periférico son independientes y paralelos
- **Técnicas de E/S:** grado de participación de la CPU en las operaciones de E/S

E/S programada

E/S por interrupciones

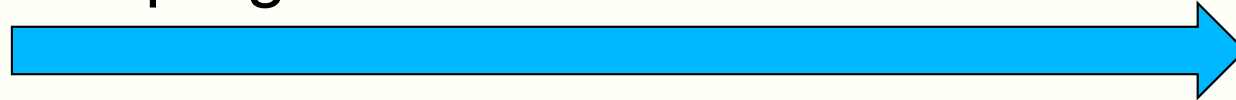
E/S por DMA



Técnicas de E/S

OCUPACIÓN de la CPU

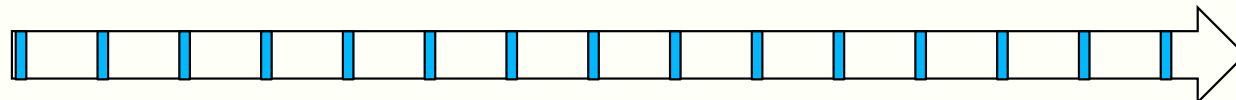
E/S programada



E/S por interrupciones



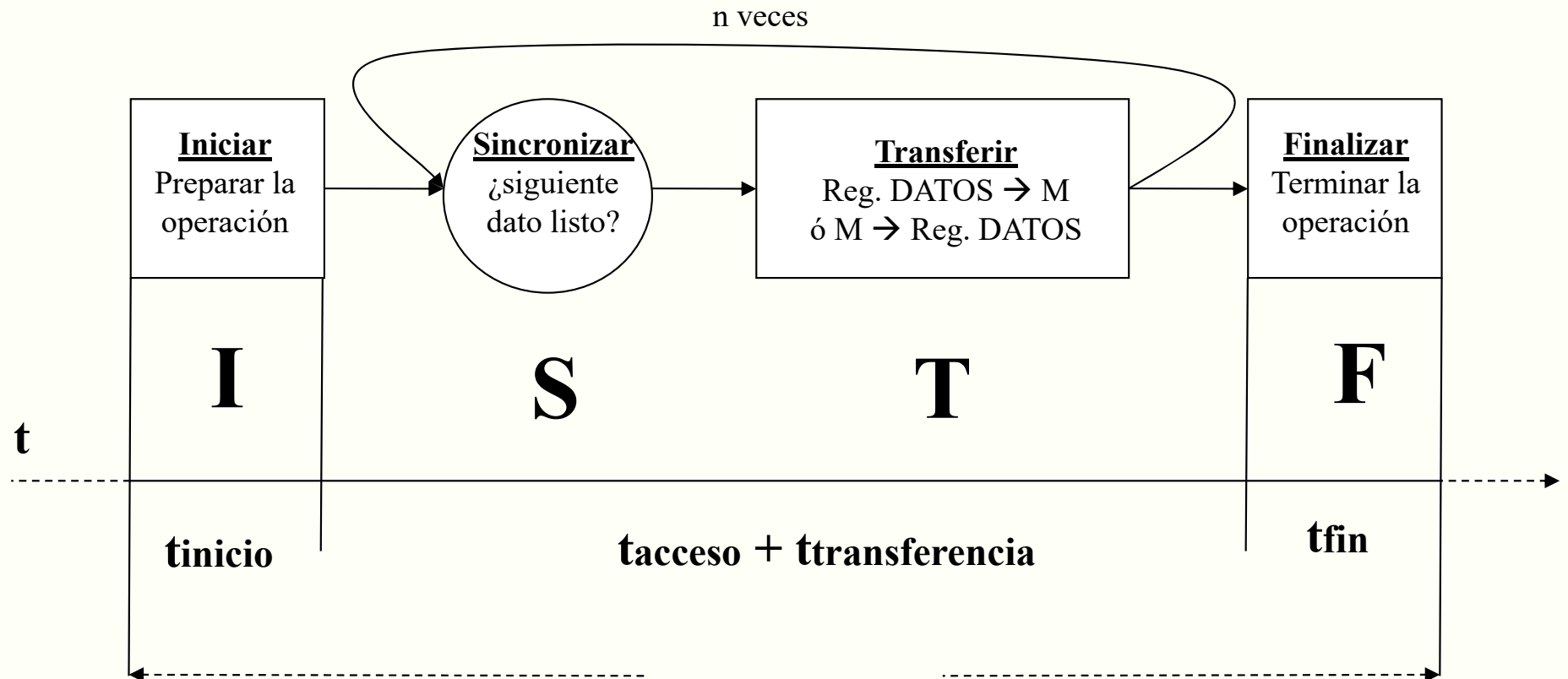
E/S por DMA



t

Técnicas de E/S

- Operación de E/S: transf. de un bloque de n datos



Disco HDD

$$t_{acc} = 5ms$$

$$CPU \ 3 \ GHz \rightarrow 3 \cdot 10^{-9} Hz = 0,3ns/ciclo$$

$$Se \ instr \ ejecuta \ 1 \ inst/ciclo \Rightarrow 3 \ inst/ns$$

$$5ms \rightarrow 5 \cdot 10^6 ns \rightarrow 5 \cdot 10^6 \cdot 3inst = 15 \text{ Millons Instruccions} \quad \begin{matrix} (\text{programas hacen mucho}) \\ (\text{es inadmisible}) \end{matrix}$$

Técnicas de E/S

Técnica	F A S E			
	I	S	T	F
-Programada	CPU	CPU	CPU	CPU
-Interrupciones				

Técnicas de E/S

Técnica	F A S E			
	I	S	T	F
- Programada	CPU	CPU	CPU	CPU
- Interrupciones	CPU	Mód	CPU	CPU
- DMA				

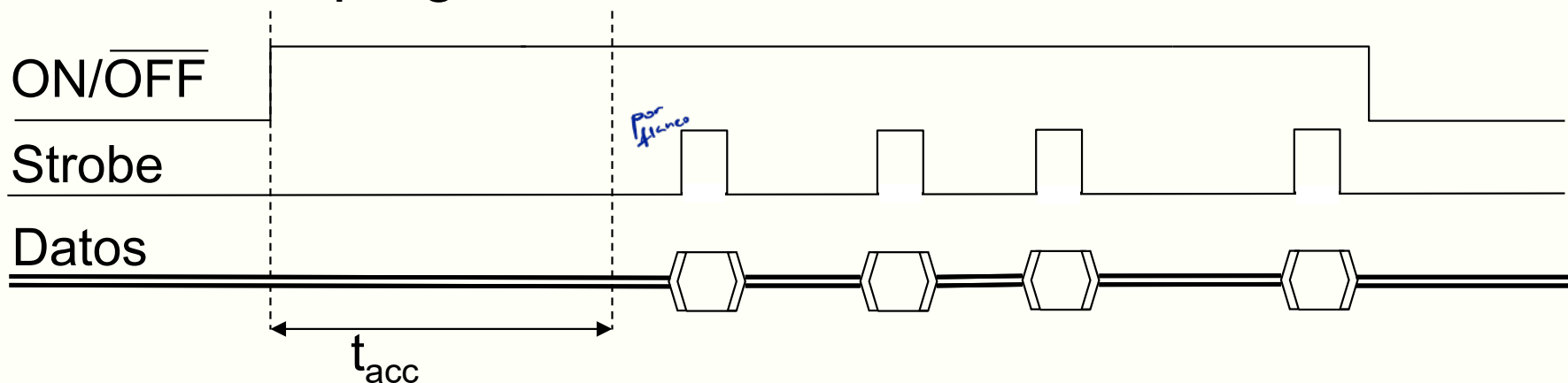

 especialmente
 lento, se
 delega al Msd

Técnicas de E/S

Técnica	F A S E			
	I	S	T	F
-Programada	CPU	CPU	CPU	CPU
-Interrupciones	CPU	Mód	CPU	CPU
-DMA	CPU	Mód	Mód	CPU

E/S programada o directa

- Todas las fases realizadas por la CPU
 - La sincronización se realiza mediante instrucciones en un bucle de espera.
-
- Ejemplo de un periférico muy simple:
 - Una vez que se activa, empieza a transmitir datos hasta que se le da la orden de parar
 - Se programará su *driver*



E/S programada o directa

Preparación o configuración

- Tamaño del bloque: n palabras
- Dir. de almacenamiento en memoria: dir_alm_M

Direcciones de E/S

- Reg. de Control  Reg. de Estado: Dir_CE [xxxxxxxx0]^{As}
- Reg. de Datos: Dir_Datos [xxxxxxxx1]_{As}

Mandatos (o 'comandos')

- Activar: ON [xxxxxxxx1]
- Desactivar: OFF [xxxxxxxx0]

Estado

- Nuevo dato listo: $LISTO$ [xxxxxxxx1]

(funciona en strobe)

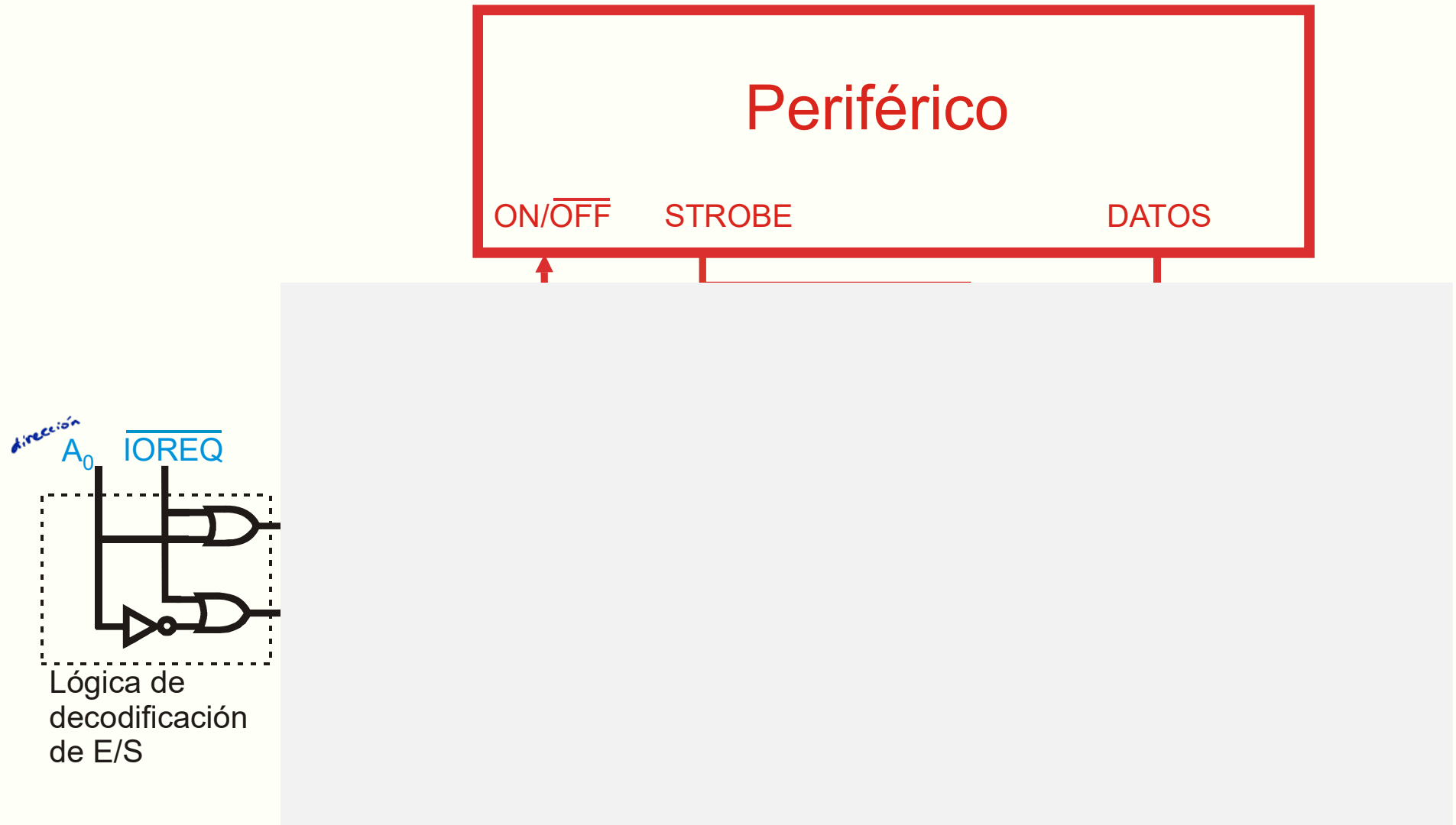
E/S programada o directa

Módulo de E/S + *driver* para E/S programada



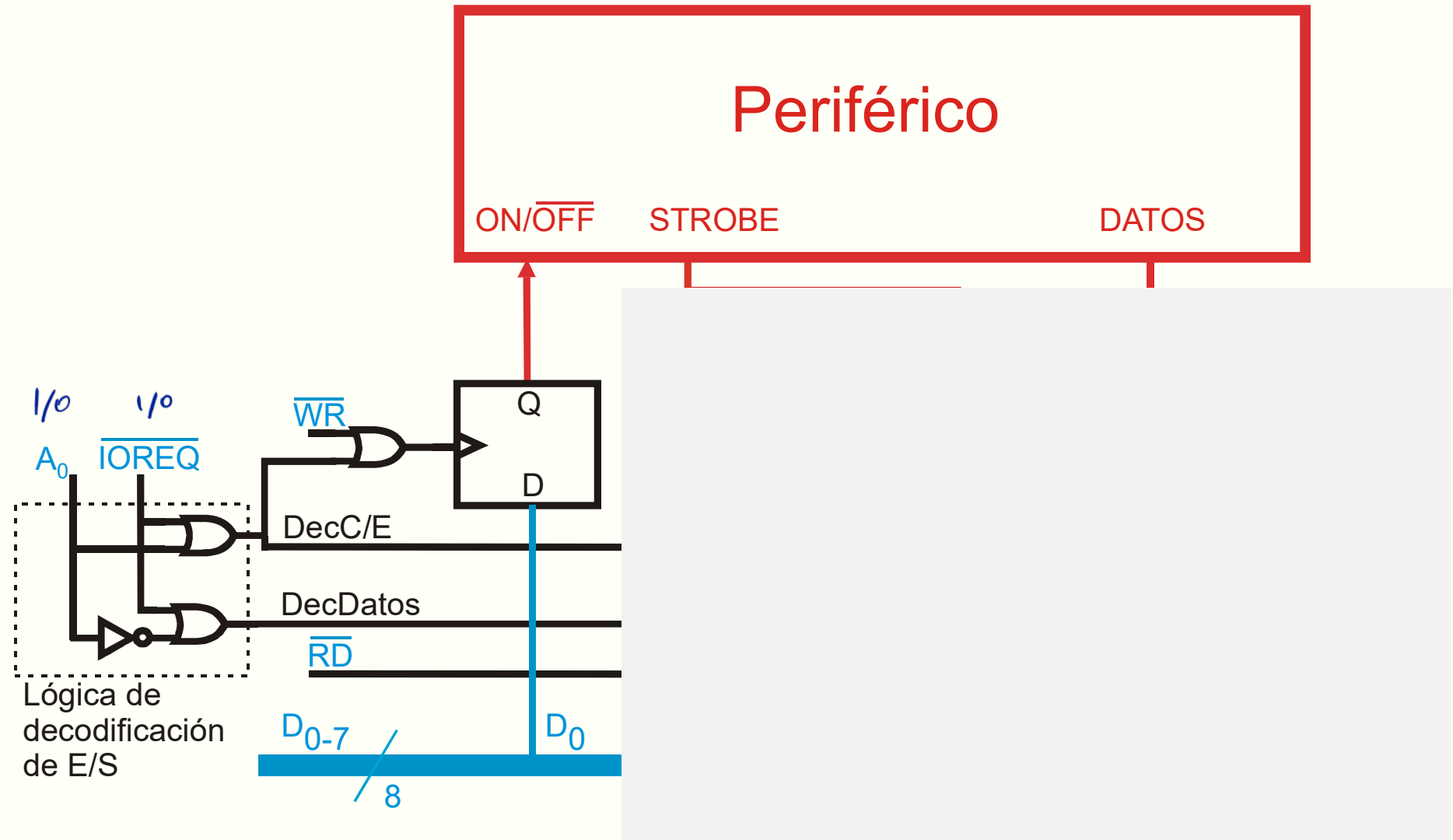
E/S programada o directa

Módulo de E/S + *driver* para E/S programada



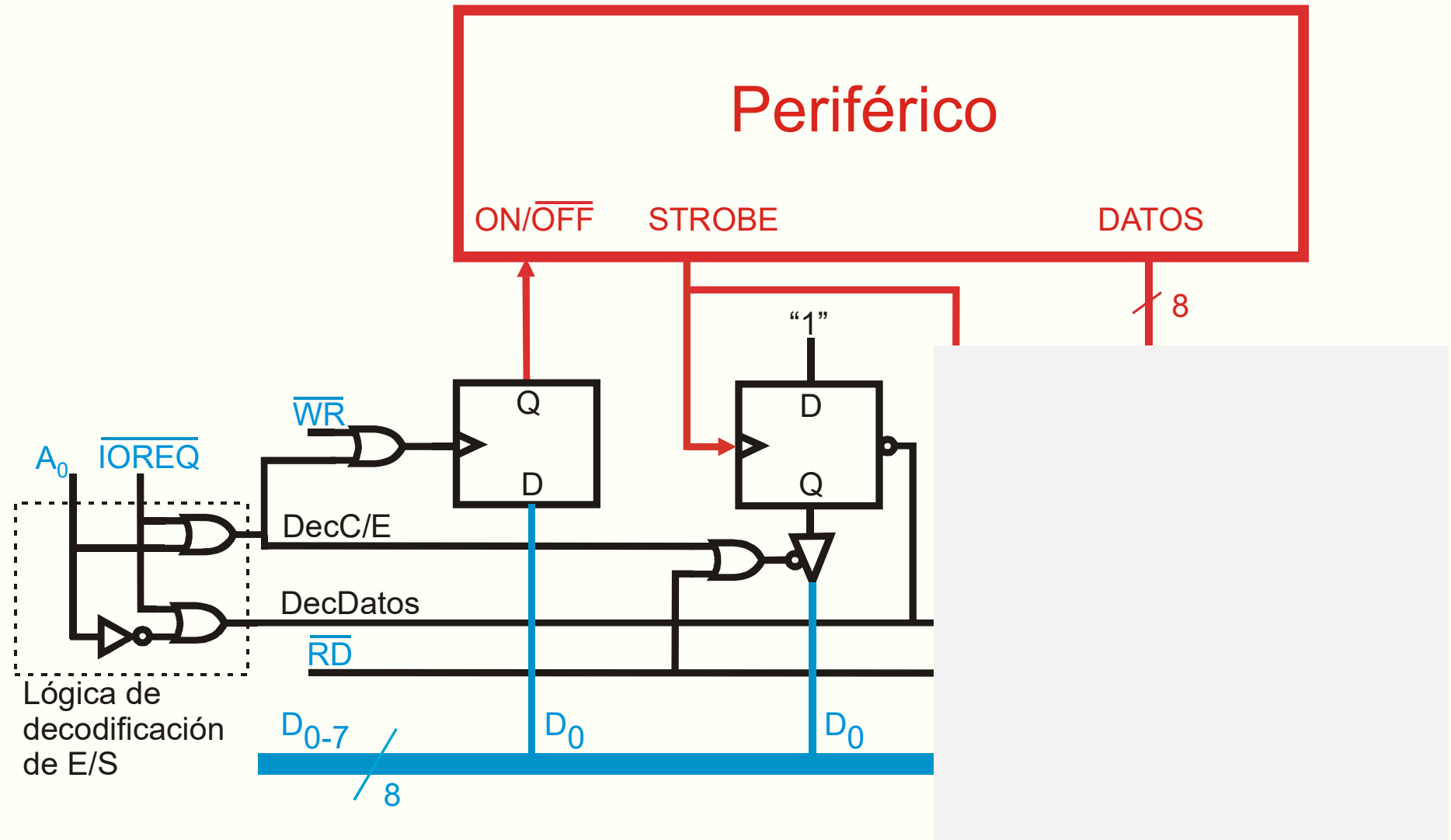
E/S programada o directa

Módulo de E/S + *driver* para E/S programada



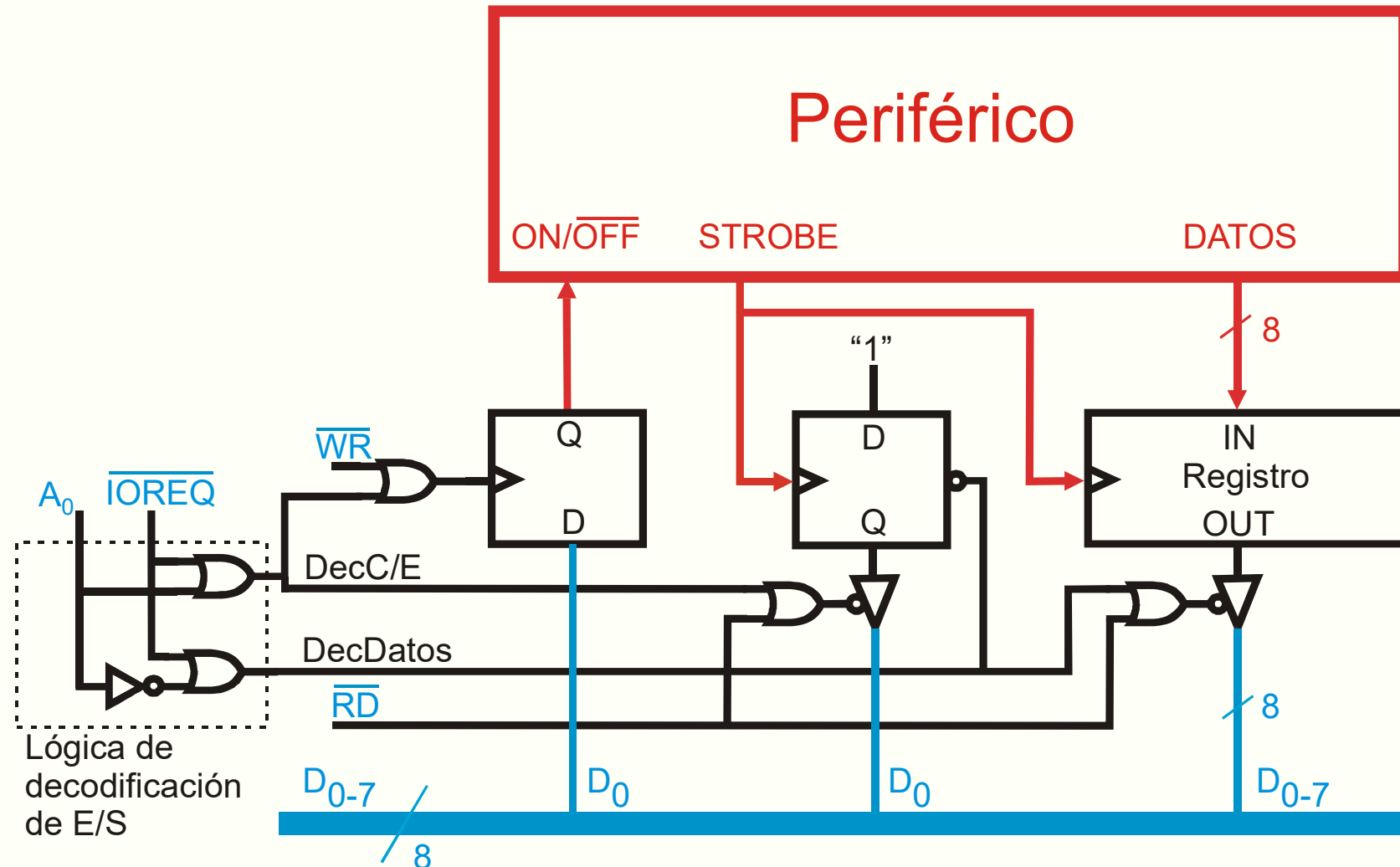
E/S programada o directa

Módulo de E/S + *driver* para E/S programada



E/S programada o directa

Módulo de E/S + *driver* para E/S programada

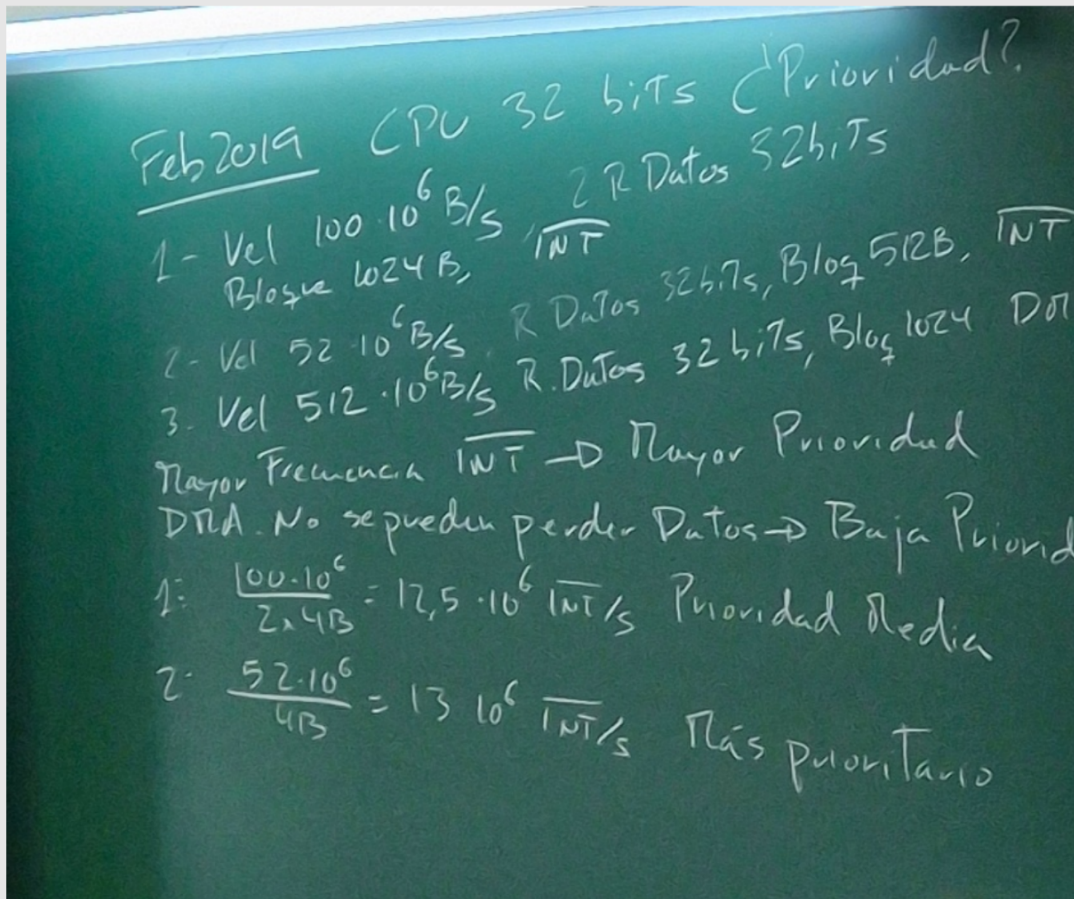


2 (1 punto) Considere un computador de 32 bits al que están conectados tres periféricos con las características básicas siguientes:

- Velocidad de transferencia: 100×10^6 B/s. Registro de datos: 2×32 bits. Bloques de 1.024 Bytes. Funciona por interrupciones.
- Velocidad de transferencia: 52×10^6 B/s. Registro de datos: 32 bits. Bloques de 512 Bytes. Funciona por interrupciones.
- Velocidad de transferencia: 512×10^6 B/s. Registro de datos: 32 bits. Bloques de 1.024 Bytes. Funciona por DMA.

Justifique numéricamente, a partir de estos datos, a cuál de los periféricos debería asignarse una mayor prioridad en cuanto a la atención a interrupciones y a cuál la menor prioridad.

WUOLAH



SOLUCIÓN

La solución más adecuada consiste en asignar la máxima prioridad al periférico que interrumpe con mayor frecuencia.

El periférico 1 interrumpe cada vez que se le llena el registro de datos (cada $4+4=8$ bytes), es decir:

$$Frecuencia_{Interrupción} = \frac{100 \times 10^6 \text{ B/s}}{8B} = 12,5 \times 10^6 \text{ Hz} = 12,5 \text{ MHz}$$

El periférico 2 también trabaja por interrupciones y dispone de un registro de datos de 4 bytes:

$$Frecuencia_{Interrupción} = \frac{52 \times 10^6 \text{ B/s}}{4B} = 13 \times 10^6 \text{ Hz} = 13 \text{ MHz}$$

Por último, el tercer periférico trabaja por DMA, por lo que solo genera una interrupción cada vez que completa la transferencia de un bloque (1.024 bytes). Su máxima frecuencia de interrupción se produce cuando esté constantemente transfiriendo información, por lo que sin tener en cuenta los tiempos de inicio, de acceso, la rutina de interrupción (finalización), se alcanzaría como máximo:

$$Frecuencia_{Interrupción} < \frac{512 \times 10^6 \text{ B/s}}{1.024B} = 1/2 \times 10^6 \text{ Hz} = 500 \text{ kHz}$$

Si se utilizaran en el cálculo los tiempos que se no se han tenido en cuenta, la frecuencia de interrupción sería incluso menor. En consecuencia la prioridad máxima se debería asignar al periférico 2 (13MHz) y la mínima al periférico 3 (500kHz).

E/S programada o directa

```
LD .R1, #dir_alm_M
LD .R2, #n
LD .R0, #H'01; ON
OUT .R0, /Dir_CE
```

```
sig: IN .R0, /Dir_CE
AND .R0, #H'01
CMP .R0, #LISTO
BNE $sig
```

leemos y guardamos en R0 al reg de estado
Hago AND con 1
y compruebo si son iguales para saber si es.
repito todo hasta q sea un 1.

```
IN .R0, /Dir_Datos
ST .R0, [.R1++]
DEC .R2
BGT $sig
```

10 inst.

```
LD .R0, #H'00; OFF
OUT .R0, /Dir_CE
```

I

S

T

F

este bucle se pasa al modulo, no le ejecuta la CPU. Es muy lento

$$\text{CPU} \quad 10 \text{ MIPS} \rightarrow \frac{10^9 \text{ ns/s}}{10 \cdot 10^6 \text{ I/s}} \rightarrow 100 \text{ ns/I}$$

$$\text{Vel} \quad 10000 \text{ B/s} \rightarrow \frac{10^6 \text{ ns/s}}{10000} \rightarrow 100 \text{ ns/B}$$

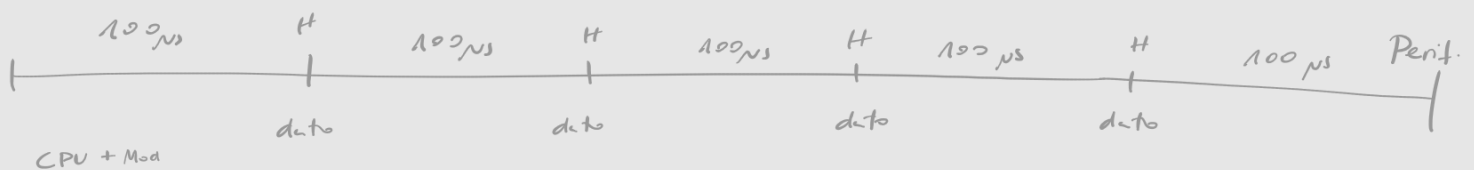
$t_{acc} = 0$ Bloque 100 datos

$$\text{Inicio} : 4 \cdot 100 \text{ ns} = 400 \text{ ns}$$

$$\text{Sinc. + Transf.} : 100 \text{ datos} \cdot 100 \text{ ns} = 10000 \text{ ns}$$

$$\text{Finalizaci3n} : 10 \text{ inst.} \cdot 100 \text{ ns} = 1000 \text{ ns}$$

$$T_{\text{total}} = t_i + t_s + t_{\text{ult. dato}} = 400 + 100000000 + 1000 = \underline{10001400 \text{ ns}}$$



No afecta al tiempo de transf.

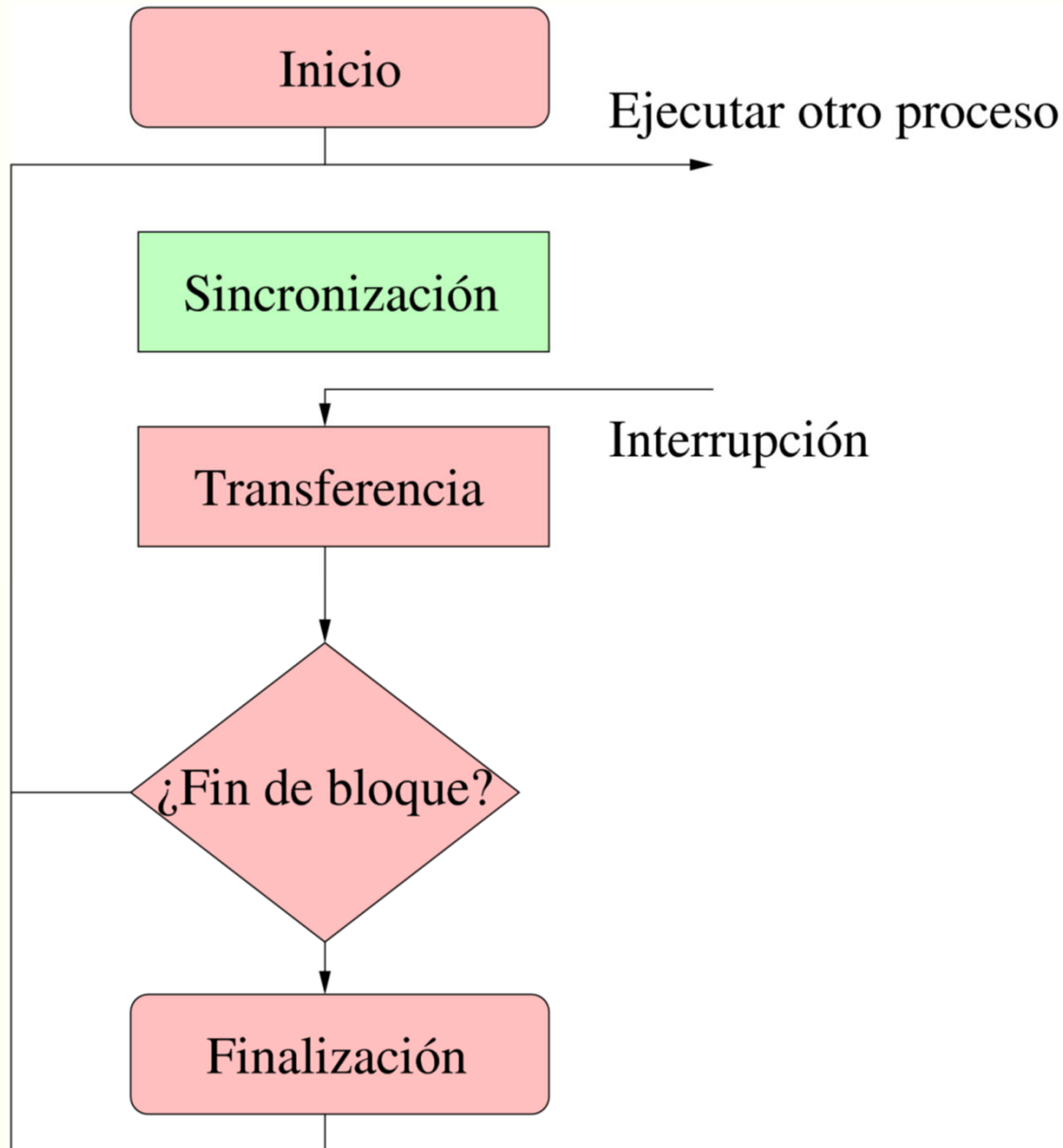
$$\text{transf.} : (4 \text{ inst.} \cdot 100 \text{ ns}) \cdot 100 \text{ datos} = \underline{40000 \text{ ns}}$$

$$\text{sinc. + transf.} : 10 \cdot 10^6 \text{ ns}$$

sinc : 99,6 % se lleva la mayor parte del tiempo. Depende mucho del perif3rico > esto es muy lento.

$$\text{transf.} : 0,4 \% \quad \frac{10^7 \text{ T+s}}{4 \cdot 10^4 \text{ T}}$$

Entrada/salida por interrupciones

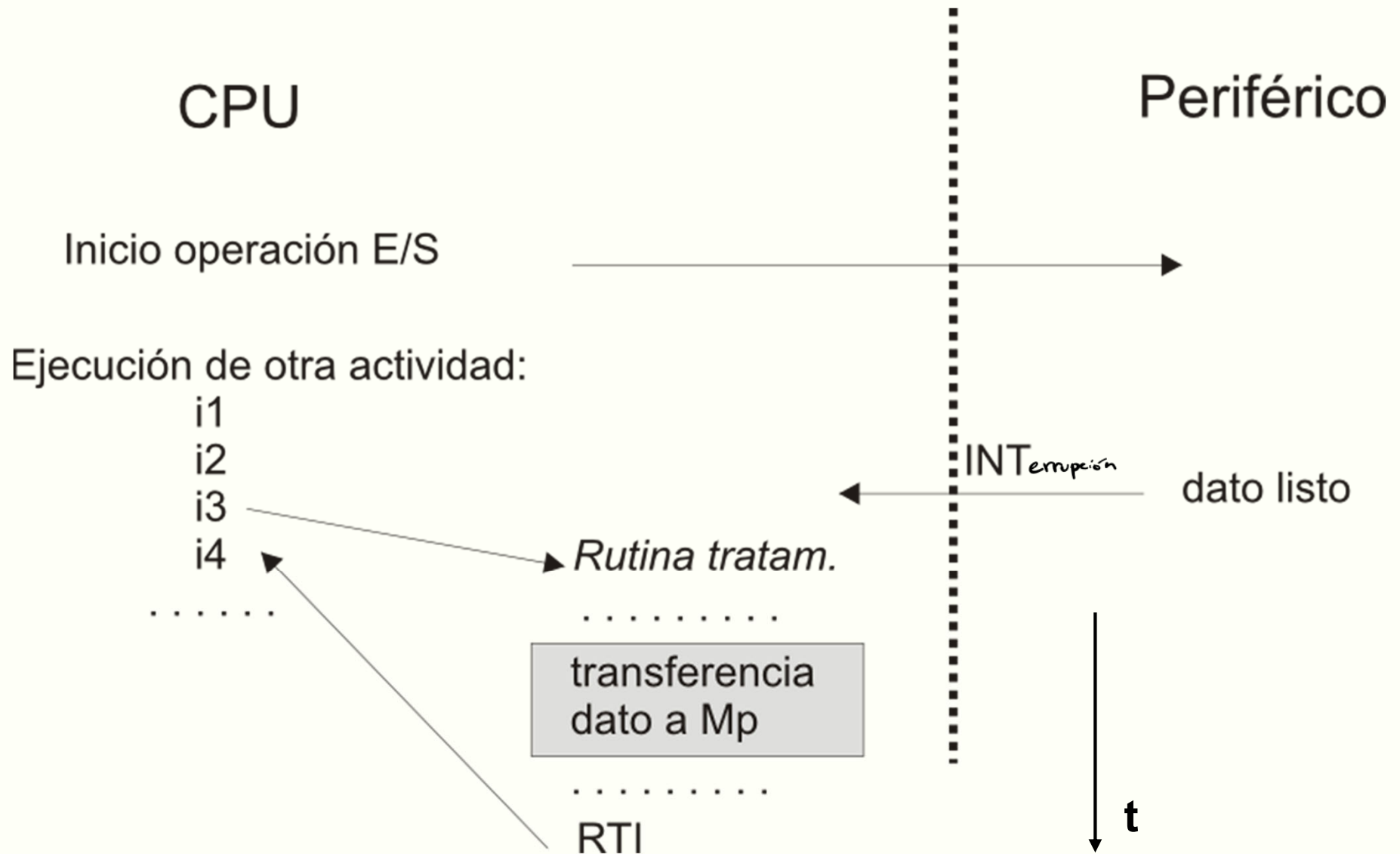


La CPU no se encarga de la sincronización

El módulo avisa a la CPU cuando está listo para una nueva transferencia

Se ahorra mucho tiempo de CPU que se usa para ejecutar otros programas

Entrada/Salida mediante interrupciones



Interrupciones

- Suceso **asíncrono** que hace que la CPU no ejecute la instrucción a la que apunta el PC
- Pasa a ejecutar la llamada *rutina de servicio de interrupción (RSI/RTI)*, que realiza la **transferencia**
 - La última vez ejecuta también la **finalización**
- Después ha de reanudar el programa interrumpido donde lo dejó (y en las condiciones en que lo dejó)
- Este mecanismo no puede afectar al comportamiento lógico de los programas
- Implica **salvar** y **restaurar** el estado del programa *en ejecución*, ya que realmente se llama a una subrutina (en instante *indeterminado*)

Interrupciones, excepciones y subrutinas

Suceso	Origen	Activación	Tratamiento
Subrutinas	Interno	Síncrona	Continuar
Excepciones	Interno	Síncrona	Continuar/Cancelar
Interrupciones	Externo	Asíncrona	Continuar

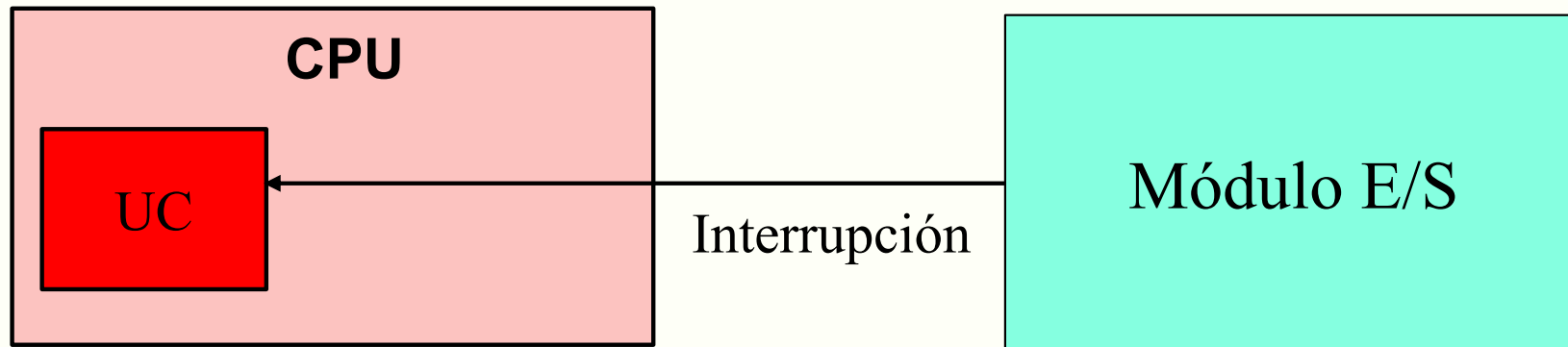
Interrupciones:

- El origen es el módulo de entrada/salida, que tiene una **temporización** propia, **determinada por el periférico**
- Pueden suceder en cualquier instante de la ejecución **de una instrucción**

Entrada/salida mediante interrupciones

Operación con un único periférico

Solicitud de interrupciones

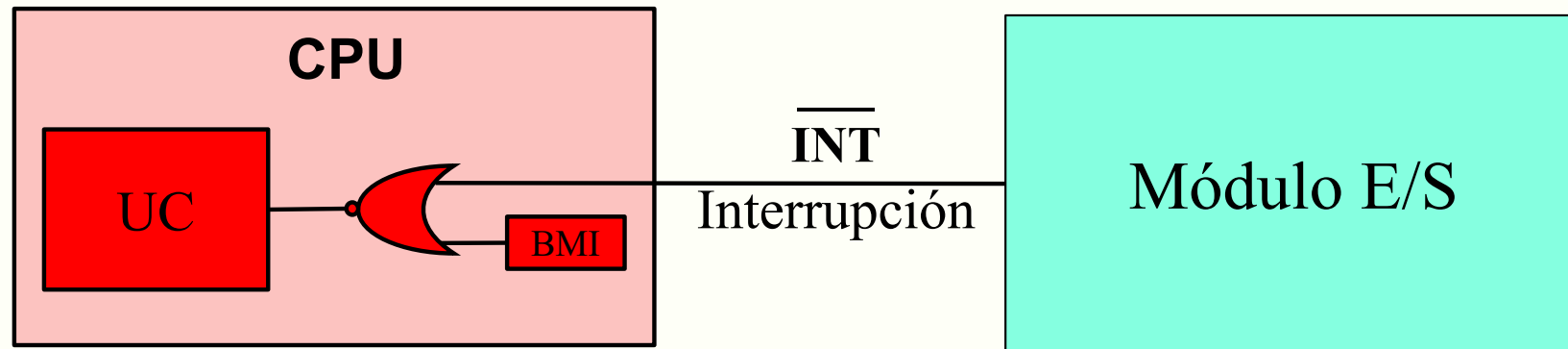


- Mediante una nueva señal de entrada a la unidad de control
- El módulo la **mantiene activa** hasta que la interrupción es atendida
- La unidad de control secuencía un conjunto de operaciones elementales para servir o tratar la **petición** de interrupción

Servicio a peticiones de interrupción

- Servicio a interrupciones:
 - abandonar la ejecución del programa en curso
 - ejecutar **otro código** que dé servicio a la solicitud del módulo de entrada/salida
- Al finalizar la ejecución del código de servicio:
 - continuar con el programa interrumpido
- Atención a la petición: al terminar la ejecución de una instrucción y antes de comenzar la siguiente
(estado de la CPU *consistente y compacto*)

Biestable de máscara de interrupciones



- Hay dos programas que se ejecutan concurrentemente:
 - a) el programa interrumpido y
 - b) el que da servicio al módulo de entrada/salida
→ condiciones de carrera
- No se puede impedir que el módulo *pida* interrupciones, pero sí que la unidad de control las *observe*, pudiendo evitar así que sean atendidas

Instrucciones para habilitar e inhibir la atención a interrupciones

- Existen instrucciones (privilegiadas) para inhibir **DI** (*Disable Interrupts*) y habilitar **EI** (*Enable Interrupts*) la atención a las interrupciones
- La UC las inhibe cuanto atiende una interrupción
- El estado de las interrupciones, BMI, pertenece a la parte privilegiada del registro de estado (RE)
- Ejemplo familia x86:
 - CLI (*Clear Interrupt Enable Flag*) y STI (*Set Interrupt Enable Flag*)
 - IF (*Interrupt Flag*) es el bit 9 del *Flags Register*

Secuencia de reconocimiento de interrupciones (SRI)

- La UC (**Hw**) lleva a cabo la **SRI** al final de cada instrucción si *observa* la línea de petición de interrupción activa:
 - Se hace siempre antes de la secuencia de *fetch*
 - Ha de salvar el estado necesario para posteriormente reanudar el programa interrumpido: **no** es un “cambio de contexto”
 - La siguiente instrucción a ejecutar ha de ser la primera de la rutina de servicio de interrupciones (se debe cargar su dirección en el PC)
- ¿Cómo se obtiene la Dirección de la RTI? Es única.

Secuencia de reconocimiento de interrupciones (SRI)

FETCH: Si ($\overline{\text{INT}} \wedge \overline{\text{BMI}}$) entonces:

PUSH Salvar PC

PUSH Salvar RE

RE.BMI $\leftarrow$ 1 (Inhibir interrupciones)

RE.S $\leftarrow$ 1 (Cambiar a modo privilegiado)

PC $\leftarrow$ DRTI (Dir. De la rutina de tratamiento de int.)

Ir a *fetch*

Si no:

Ir a *fetch*

- El PC y el RE se **salvan en pila** por sencillez
- Además, **se inhiben las interrupciones** para evitar reconocer indefinidamente la misma
- Se pasa a **modo privilegiado** para poder ejecutar instrucciones de entrada/salida

Instrucción de retorno de rutina de servicio de interrupción

RETI:	Restaurar RE	<i>POP</i>
	Restaurar PC	<i>POP</i>
	Ir a <i>fetch</i>	

- Implícitamente **habilita** las interrupciones
- Esta instrucción deshace lo hecho en la secuencia de reconocimiento de interrupción. Ejemplos:
 - IRET (familia x86)
 - RETT (SPARC)
 - RTE (88110)

Rutina de servicio de interrupción (RSI o RTI)

- No debe alterar la lógica del programa interrumpido: debe **salvar y restaurar** el estado que altere, es decir, todos los registros que utilice
- Se ejecuta debido a un suceso asíncrono y externo → **no** se le pueden pasar parámetros por la pila ni en registros
- Su estado inicial *se puede definir* mediante información almacenada en direcciones absolutas (*conocidas*) de memoria
- Finaliza con una instrucción de tipo RETI

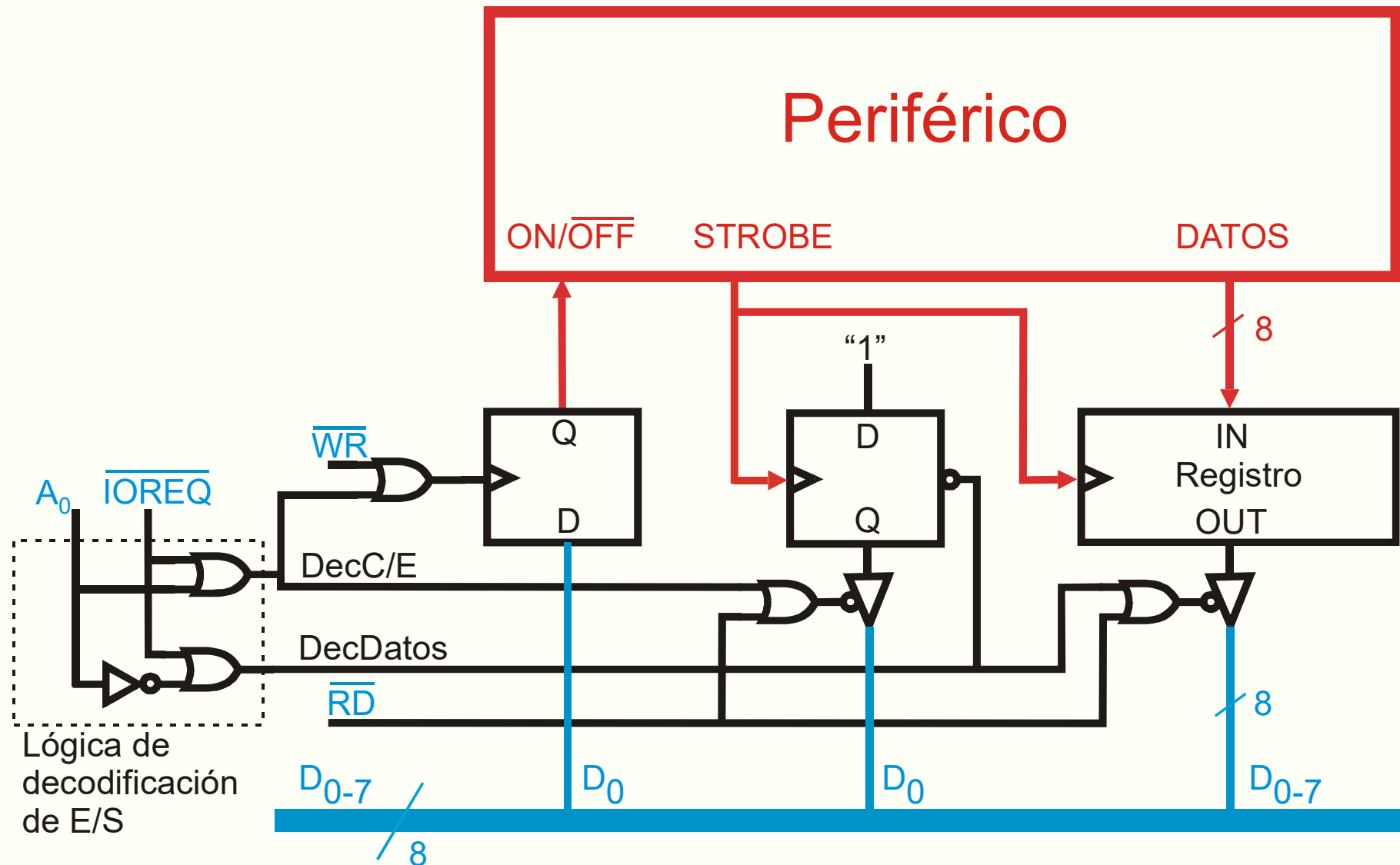
Sobrecoste de las interrupciones

- Secuencia de reconocimiento de interrupciones.
- Salvar/restaurar registros
- Acceso y actualización de la información de estado
- RETI

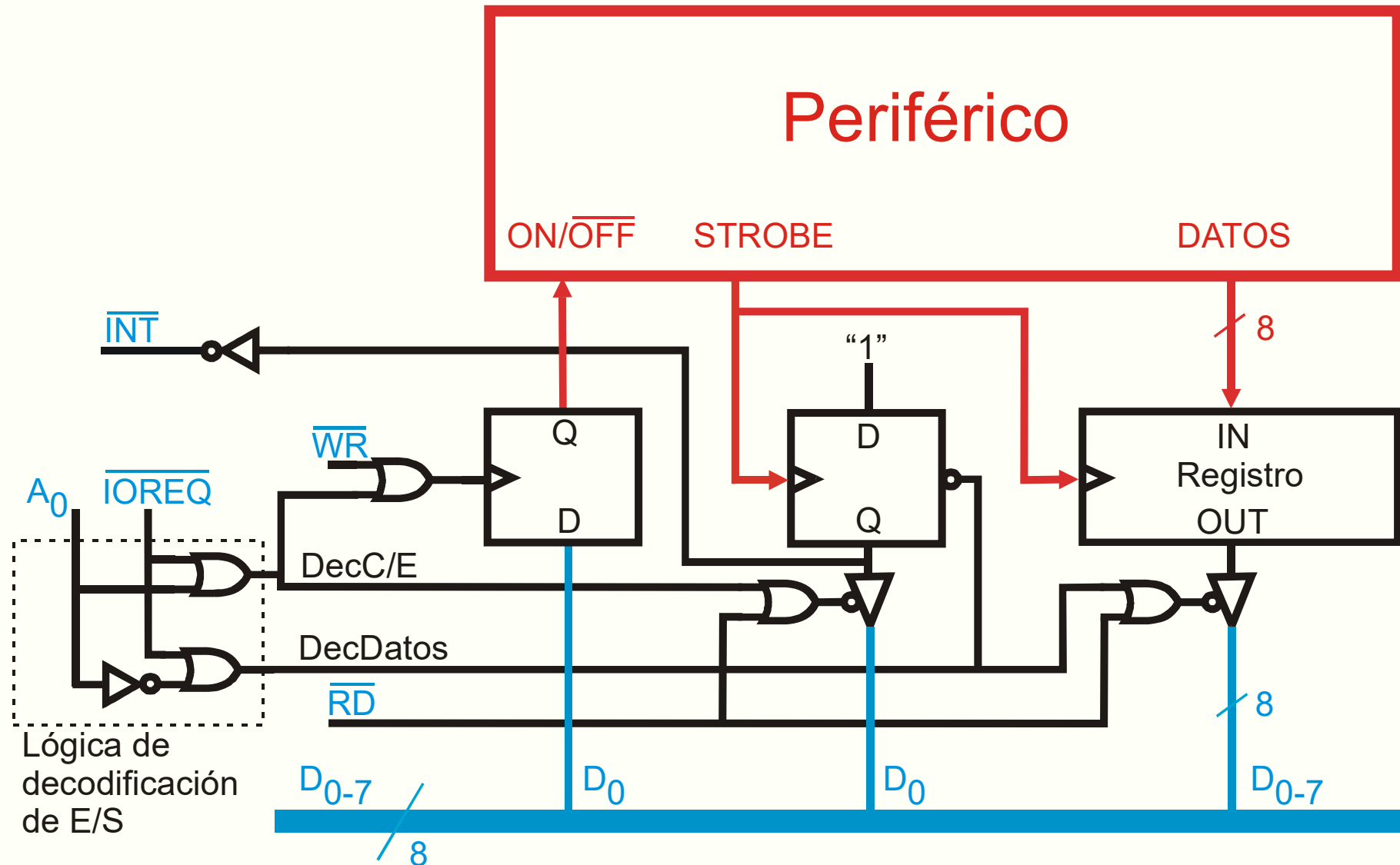
Entrada/salida mediante interrupciones

Ejemplo con un periférico simple

E/S programada o directa



E/S por interrupciones



Iniciación

```
LD .R1,#dir_alm_M
ST .R1,/Dir_dir_alm_M
LD .R2,#n
ST .R2,/Dir_contador
LD .R0,#H'01; ON
OUT .R0, /Dir_CE
;
; Llamada al S.O. para
; “que dé paso a otro proceso”
;
```

-
- Dir_dir_alm_M y Dir_contador: estado interno de la RTI (almacenado en variables globales)
 - dirección de memoria donde se almacenará el siguiente dato
 - número de datos que quedan para completar la operación

Rutina de tratamiento de interrupción

- En negro: acceso y actualización del estado interno de la RTI (sobrecoste)
- En verde: lectura del dato y almacenamiento en memoria

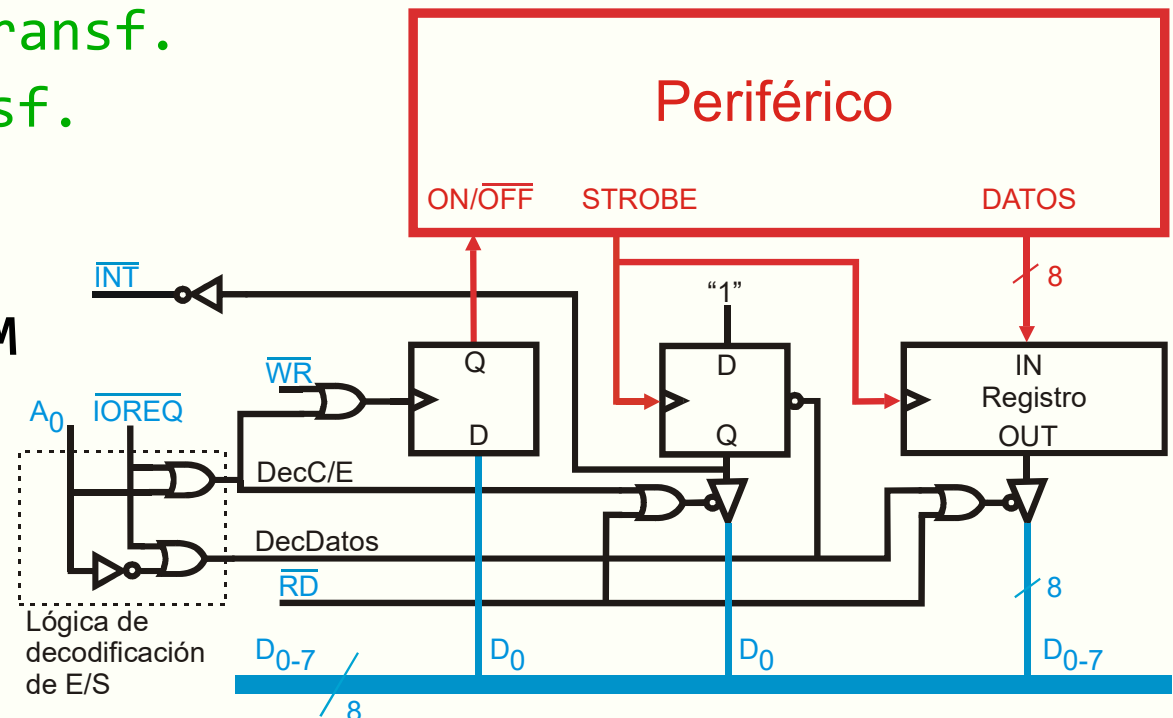
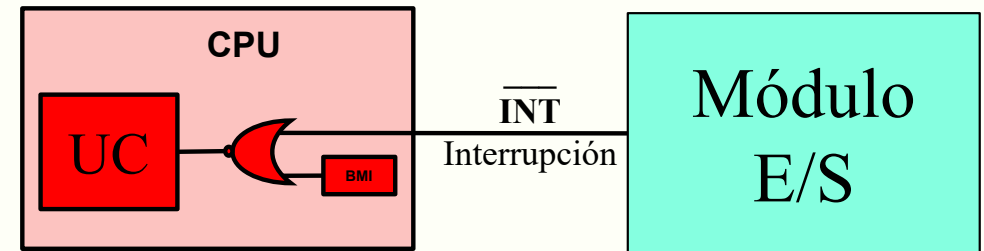
```
LD .R1,/Dir_dir_alm_M
LD .R2,/Dir_contador
IN .R0,/Dir_Datos; Transf.
ST .R0,[.R1++]; Transf.
DEC .R2
CALLZ $Fin
ST .R1,/Dir_dir_alm_M
ST .R2,/Dir_contador
```

Rutina de tratamiento de interrupción

- Se añade al sobrecoste la salvaguarda y recuperación de registros

```

DRTI: PUSH .R0
      PUSH .R1
      PUSH .R2
      LD .R1,/Dir_dir_alm_M
      LD .R2,/Dir_contador
      IN .R0,/Dir_Datos; Transf.
      ST .R0,[.R1++]; Transf.
      DEC .R2
      CALLZ $Fin
      ST .R1,/Dir_dir_alm_M
      ST .R2,/Dir_contador
      POP .R2
      POP .R1
      POP .R0
      RETI
    
```



Rutina de tratamiento de interrupción

- El sobrecoste también incluye el tiempo dedicado a la **SRI** (no visible)

- Se añade la finalización de la operación

```
DRTI: PUSH .R0
      PUSH .R1
      PUSH .R2
      LD .R1,/Dir_dir_alm_M
      LD .R2,/Dir_contador
      IN .R0,/Dir_Datos; Transf.
      ST .R0,[.R1++]; Transf.
      DEC .R2
      CALLZ $Fin
      ST .R1,/Dir_dir_alm_M
      ST .R2,/Dir_contador
      POP .R2
      POP .R1
      POP .R0
      RETI
```

```
Fin:  LD .R0,#H'00; OFF
      OUT .R0, /Dir_CE
      ;
      ; Llamada al S.O. para
      ; “anotar terminación”
      ;
      RET
```


Entrada/salida mediante interrupciones

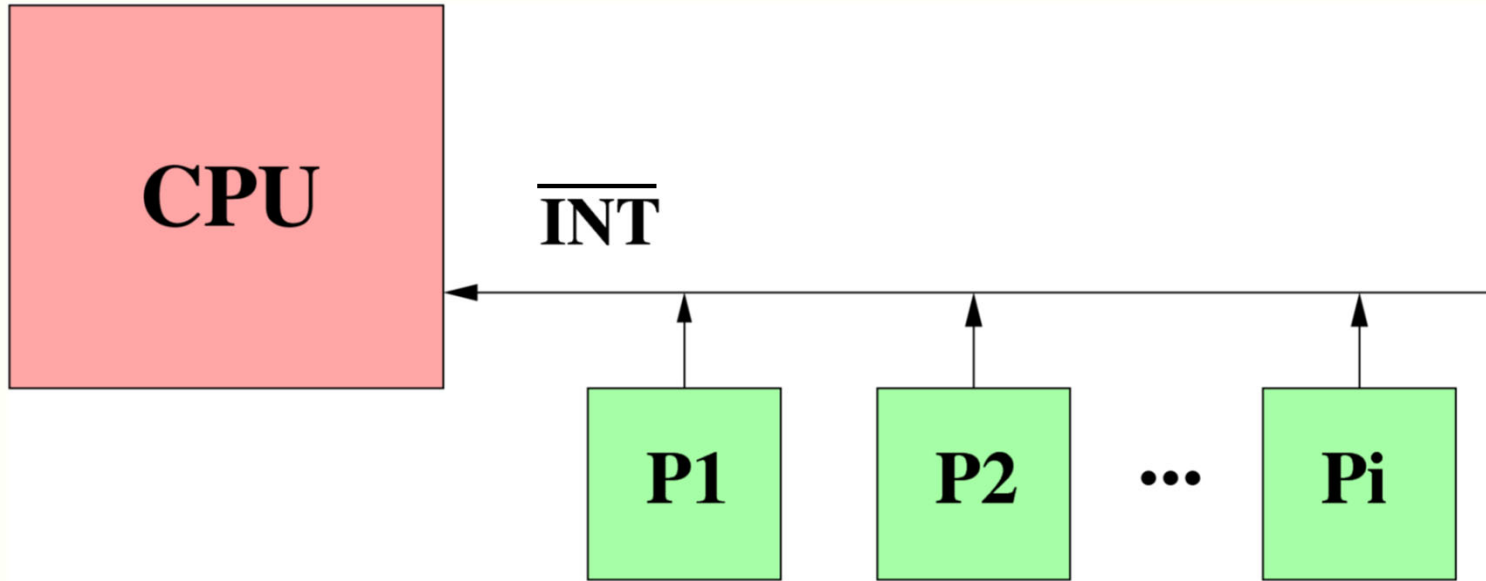
Operación con múltiples periféricos

Operación con múltiples periféricos

Problemas a resolver:

- Conexionado
- Identificación del solicitante
- Localización de la rutina de servicio para el solicitante
- Prioridades en caso de peticiones *simultáneas*
- Anidamiento de rutinas de servicio

Conexión de varios periféricos



- La línea forma un OR cableado:
 - Se activa en el momento que alguno la activa y permanece activa hasta que todos la desactiven

Identificación mediante muestreo

Sólo hay un RTI que trata todos los casos:
consulta uno a uno → implícitamente asigna prioridades

```
DRTI:  PUSH    .R1
        IN     .R1, /Dir_CE_P1
        CMP    .R1, #INT_Pend_P1
        BZ     $Rut_Espec_P1
        IN     .R1, /Dir_CE_P2
        CMP    .R1, #INT_Pend_P2
        BZ     $Rut_Espec_P2
        . . . .
```

Rut_Espec_P1: Salvar estado
Cód. específico de P1
Restaurar estado
RETI

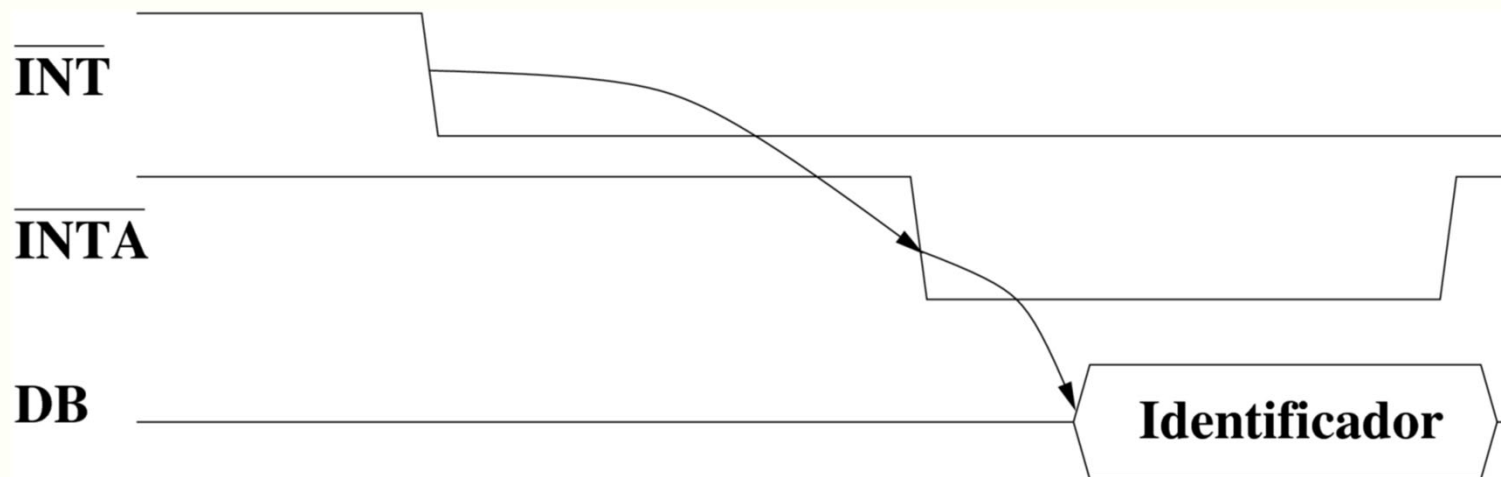
Rut_Espec_P2: Salvar estado
Cód. específico de P2
Restaurar estado
RETI

Análisis

- En caso de **peticiones simultáneas** solo se ejecuta la más prioritaria que es **la que se consulta primero**
- **No es posible el anidamiento** entre las distintas rutinas de servicio porque la interrupción ha de ser previamente reconocida para averiguar su prioridad: la más prioritaria debe esperar
- Se utilizan **muchas instrucciones** en la identificación de la interrupción cuando existen muchos periféricos

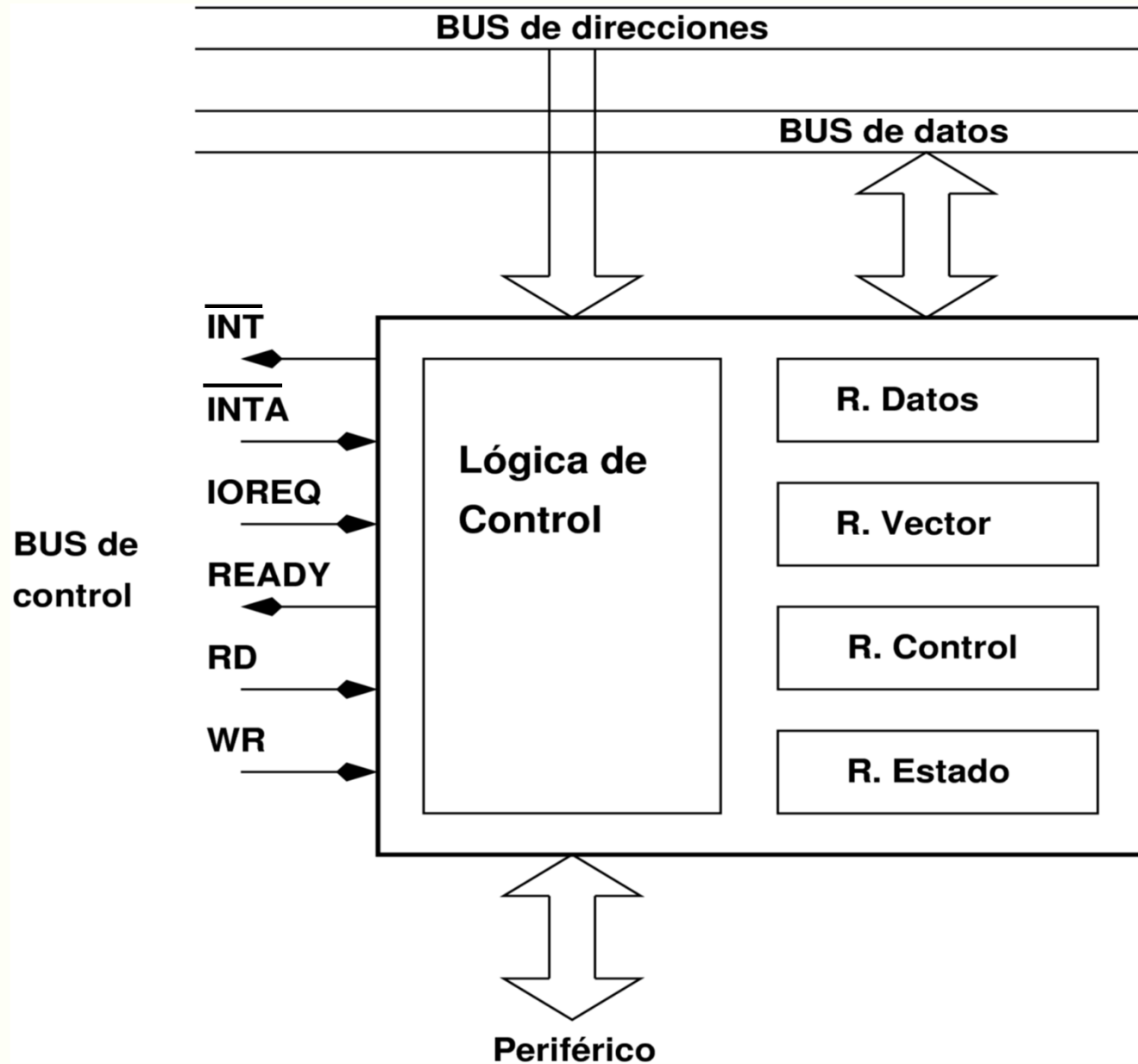
Vectorización

- Mediante el **ciclo de bus de reconocimiento de interrupciones** la CPU pide a quien solicite la interrupción (módulo E/S) que se identifique **durante la SRI**
- Nueva señal INTA (*Interrupt Acknowledge*)
- Durante la **inicialización de la operación** de E/S, el **identificador¹** se carga en un registro especial del módulo de entrada/salida (**Registro del vector de interrupción**)



-Nota1: identificador="vector de interrupción"

Módulo de entrada/salida con interrupciones vectorizadas



Secuencia de reconocimiento con vectorización

FETCH: Si $(\overline{\text{INT}} \wedge \text{RE}.\overline{\text{BMI}})$ entonces:

 Salvar PC

 Salvar RE

$\text{RE}.\text{BMI} \leftarrow 1$ (Inhibir interrupciones)

$\text{RE}.\text{S} \leftarrow 1$ (Cambiar a modo privilegiado)

 Ciclo de bus de reconocimiento de interrupciones
 (activación $\overline{\text{INTA}}$) $\text{DR} \leftarrow \text{Vector}$

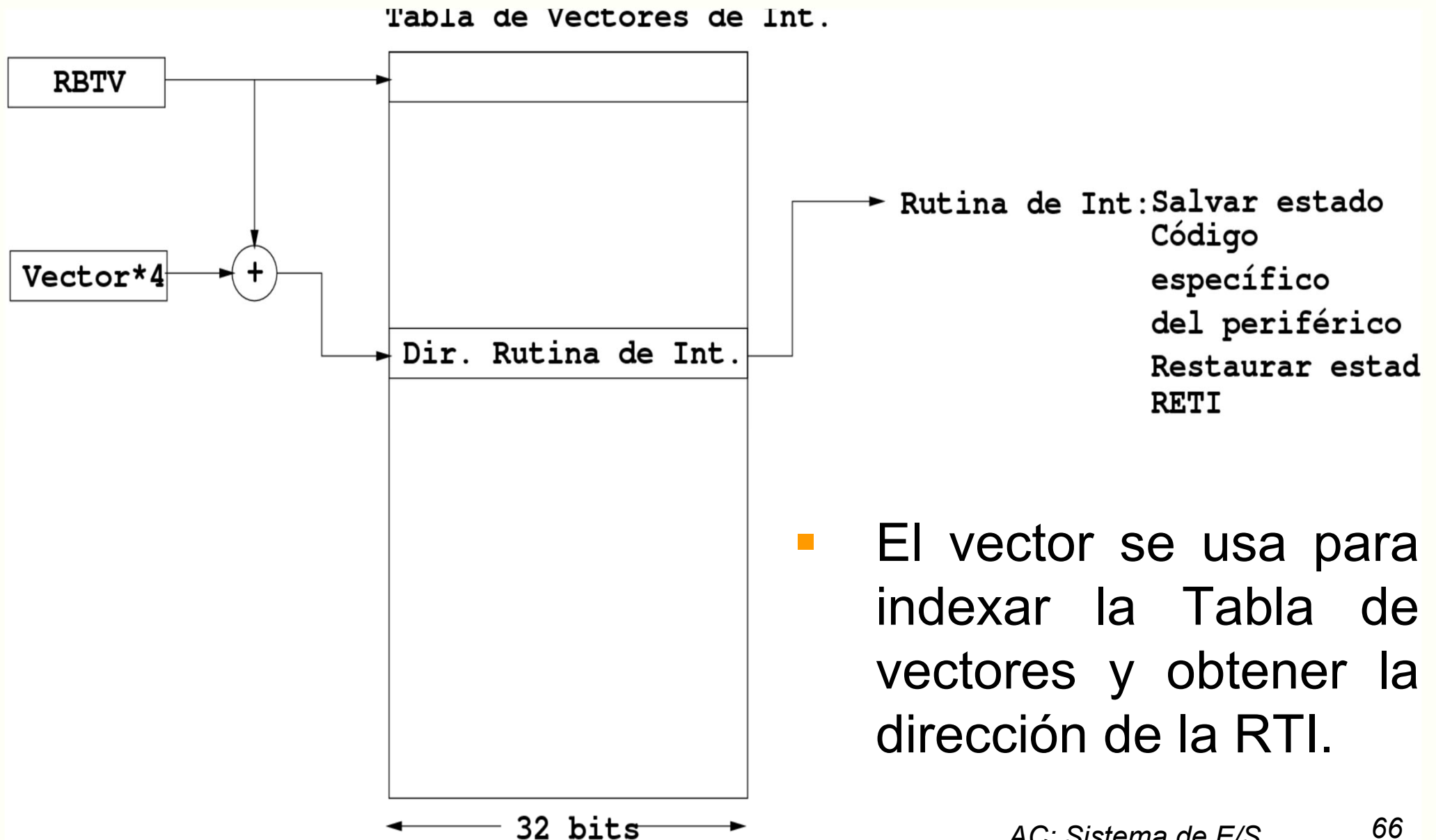
$\text{PC} \leftarrow f(\text{Vector})$

 Ir a *fetch*

Si no:

 Ir a *fetch*

Tabla de vectores de interrupción

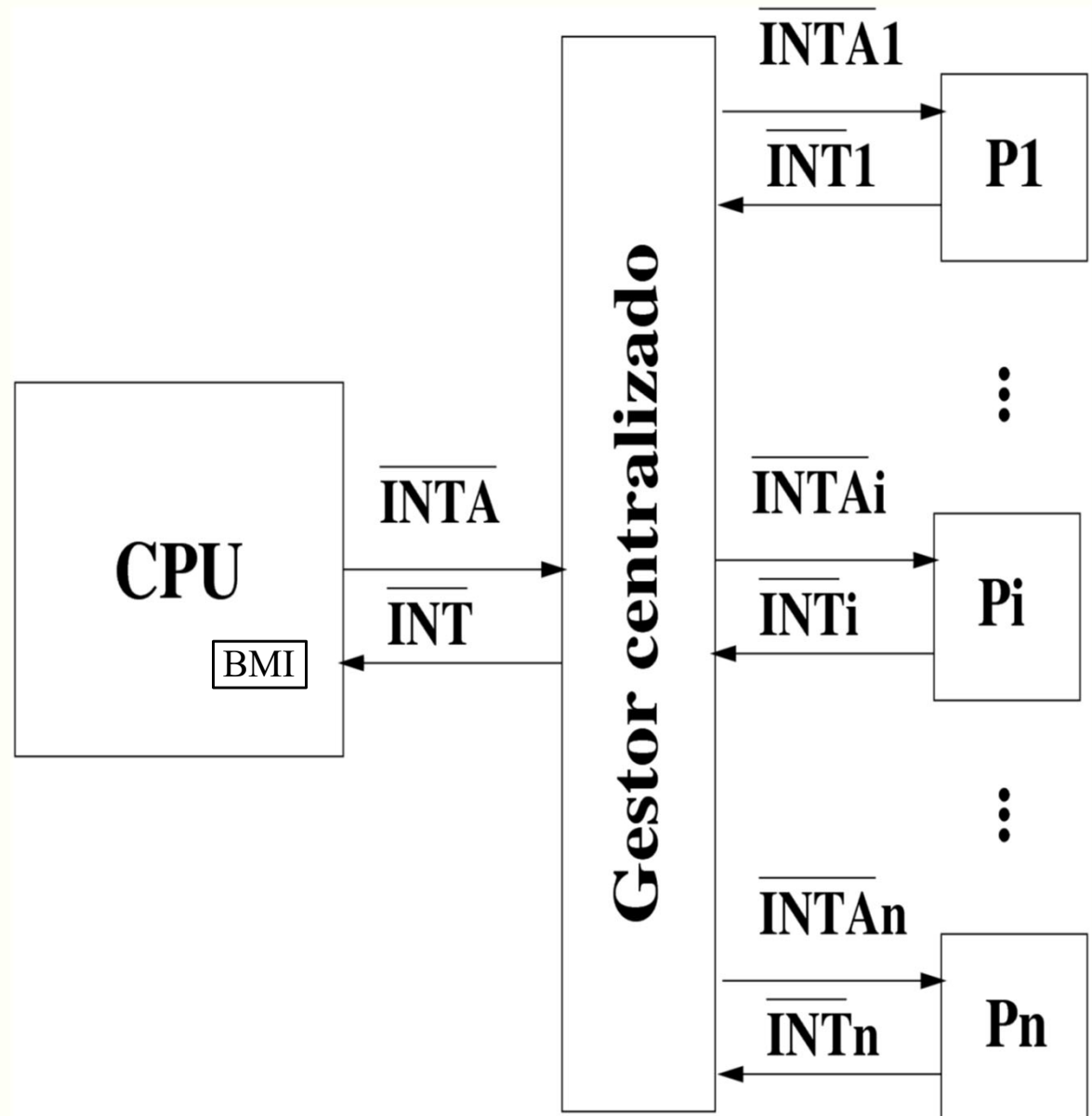


Prioridades

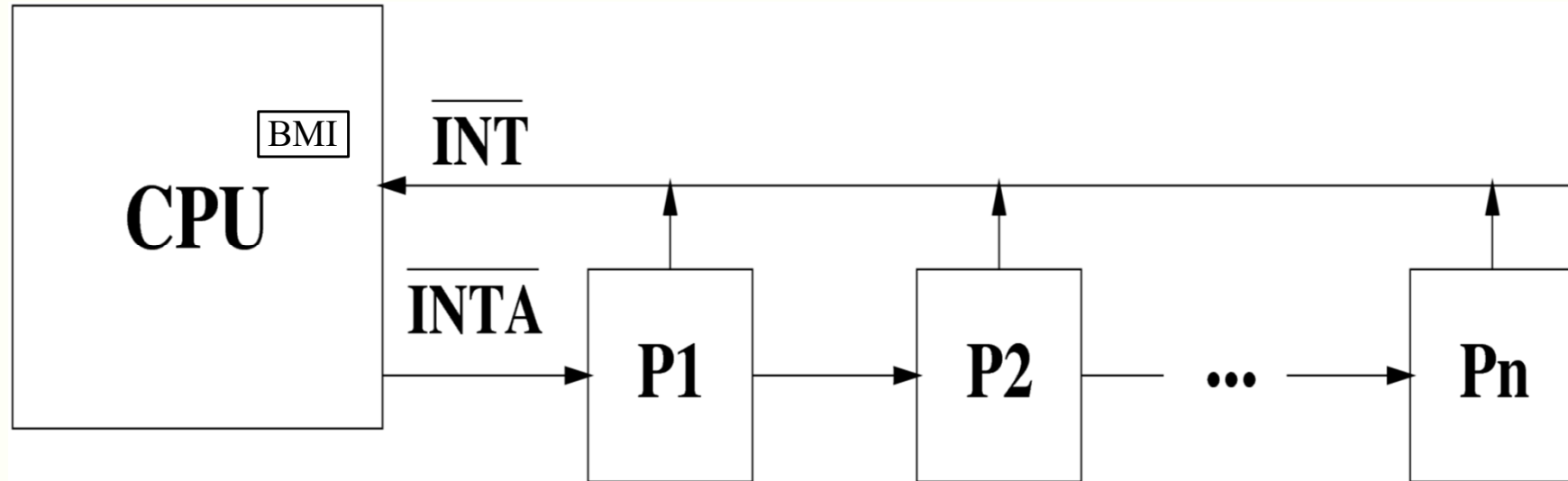
- La señal de reconocimiento le ha de llegar al más prioritario de los módulos solicitantes.
- Se necesita un esquema de prioridades hardware
- Tres opciones:
 - Gestor centralizado
 - Gestor encadenado (*daisy chain*)
 - Esquema híbrido

Gestor centralizado

- No ampliable
- Sencillo
- Ej: 8259 Intel
- Esquema de prioridades flexible

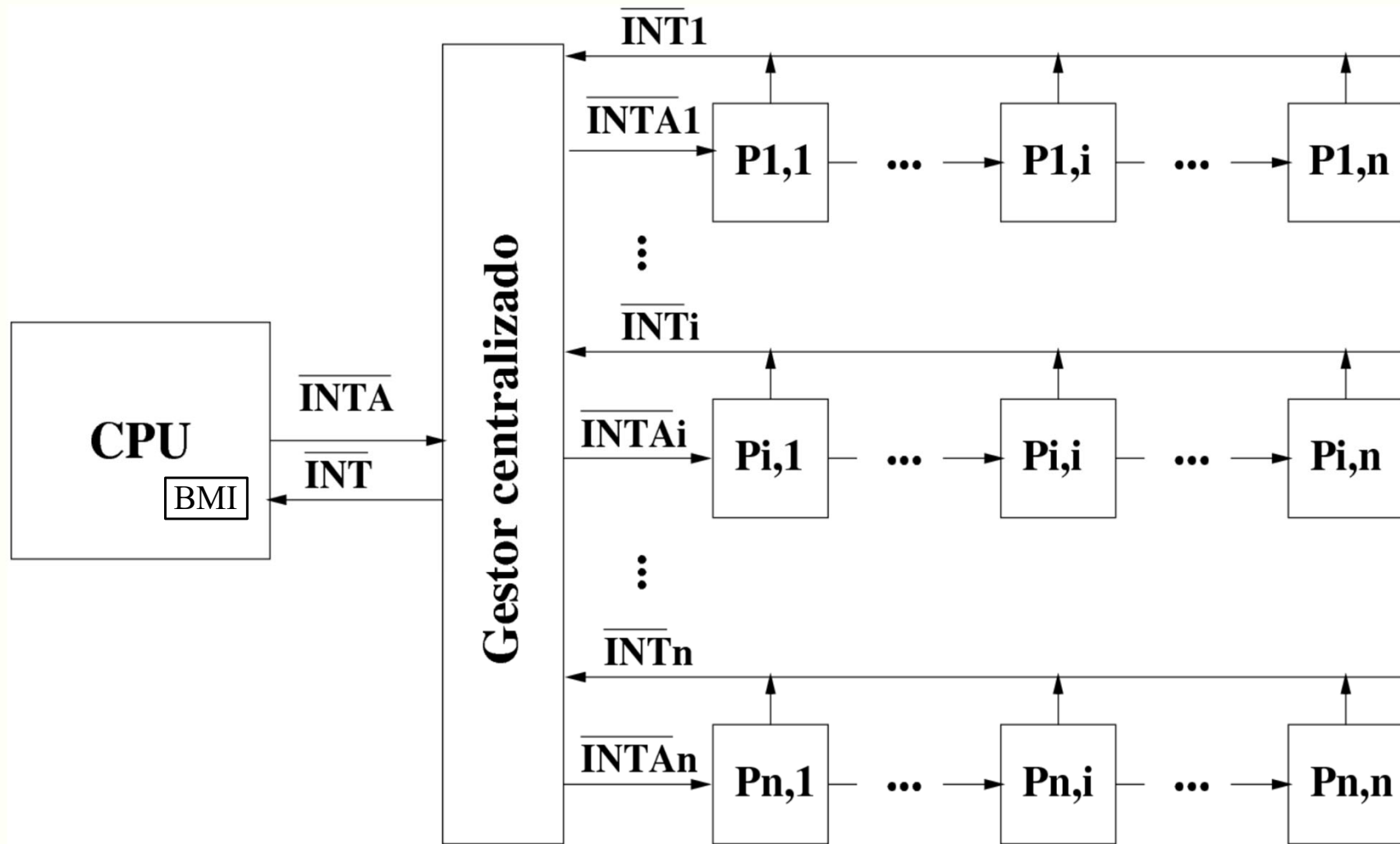


Gestor encadenado (*daisy chain*)



- El tiempo de ciclo de bus ha de ser suficiente
- Se pueden conectar tantos módulos como sea necesario (escalable, pero más lento)
- Prioridad por la posición ocupada

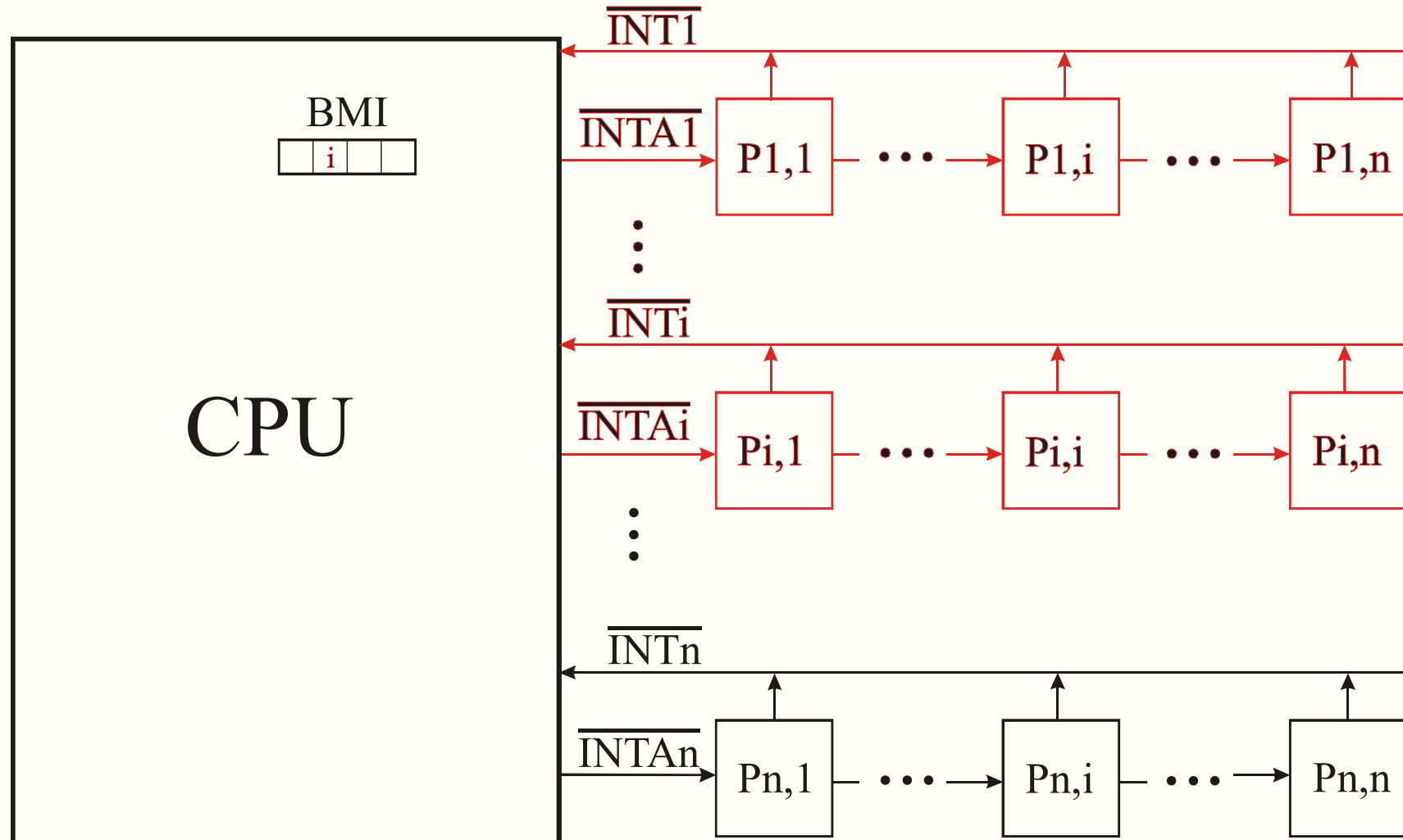
Híbrido



Análisis

- Rápida identificación del solicitante
- Conflictos de prioridades resueltos por hardware
- Anidamiento según la prioridad de líneas, pero:
 - Si sólo existe un biestable de máscara o están todas permitidas o todas inhibidas: No admite **anidamiento**
 - Con varios biestables de máscara de interrupción se puede implementar una **inhibición selectiva** por niveles, tantos bits como haga falta para codificar todas las líneas:
 - Prioridad: 1 baja, 7 alta.

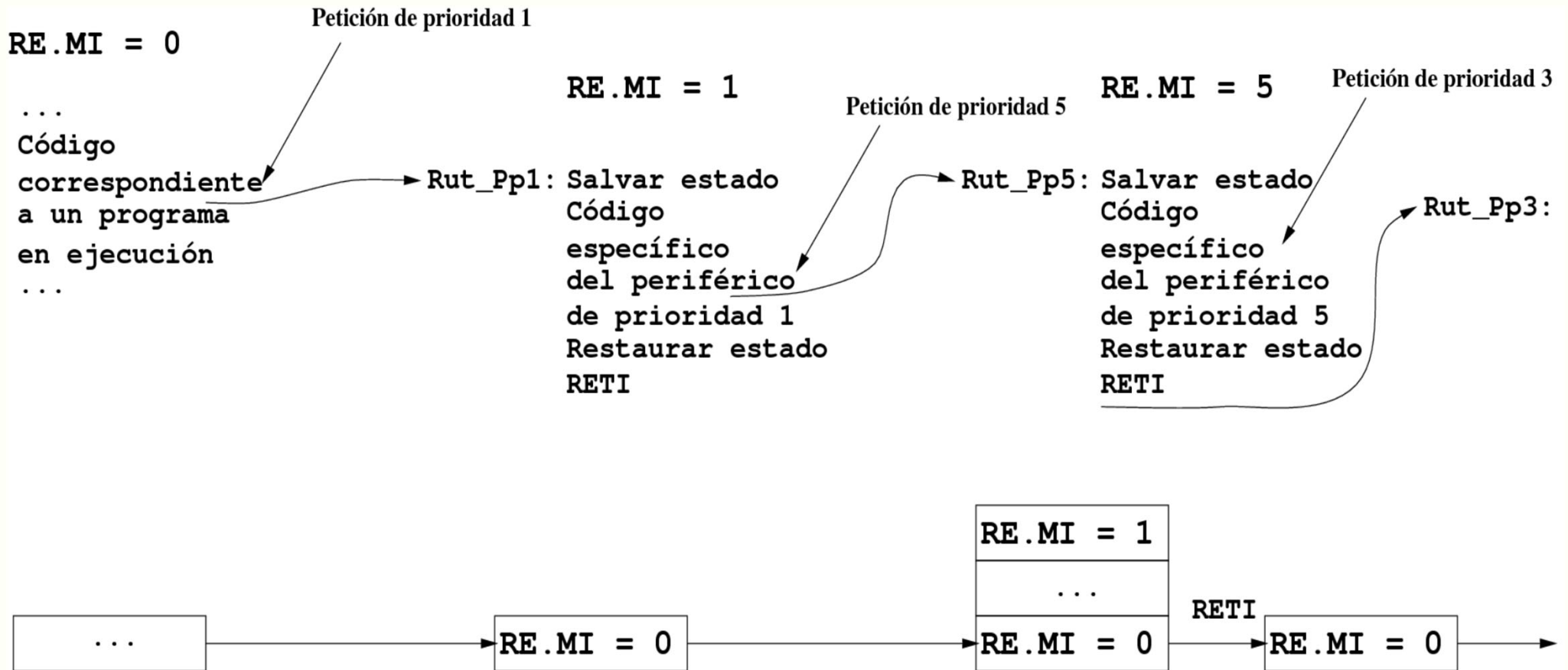
Niveles de ejecución



Secuencia de reconocimiento de interrupciones con inhibición selectiva

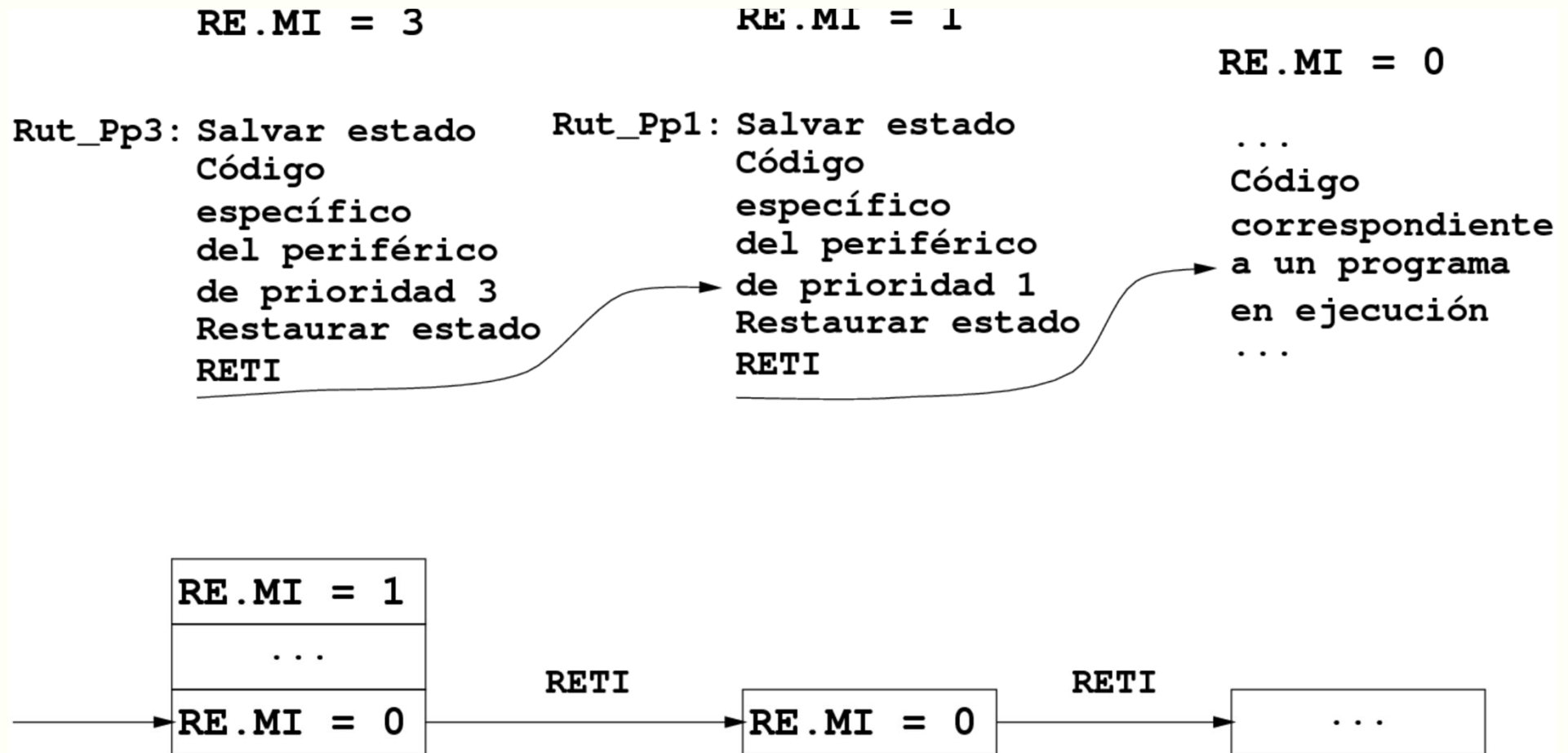
```
FETCH: Si  $(i = \max(INT_j), \forall j) \wedge (i > RE.MI)$  entonces:  
    Salvar PC  
    Salvar RE  
     $RE.MI \leftarrow i$  (Establecer nueva máscara interr..nes)  
     $RE.S \leftarrow 1$  (Cambiar a modo privilegiado)  
    Ciclo de bus de reconocimiento de interrupciones  
    (activación  $INTA_i$ )     $DR \leftarrow \text{Vector}$   
     $PC \leftarrow f(\text{Vector})$   
    Ir a fetch  
Si no:  
    Ir a fetch
```


Ejemplo de anidamiento

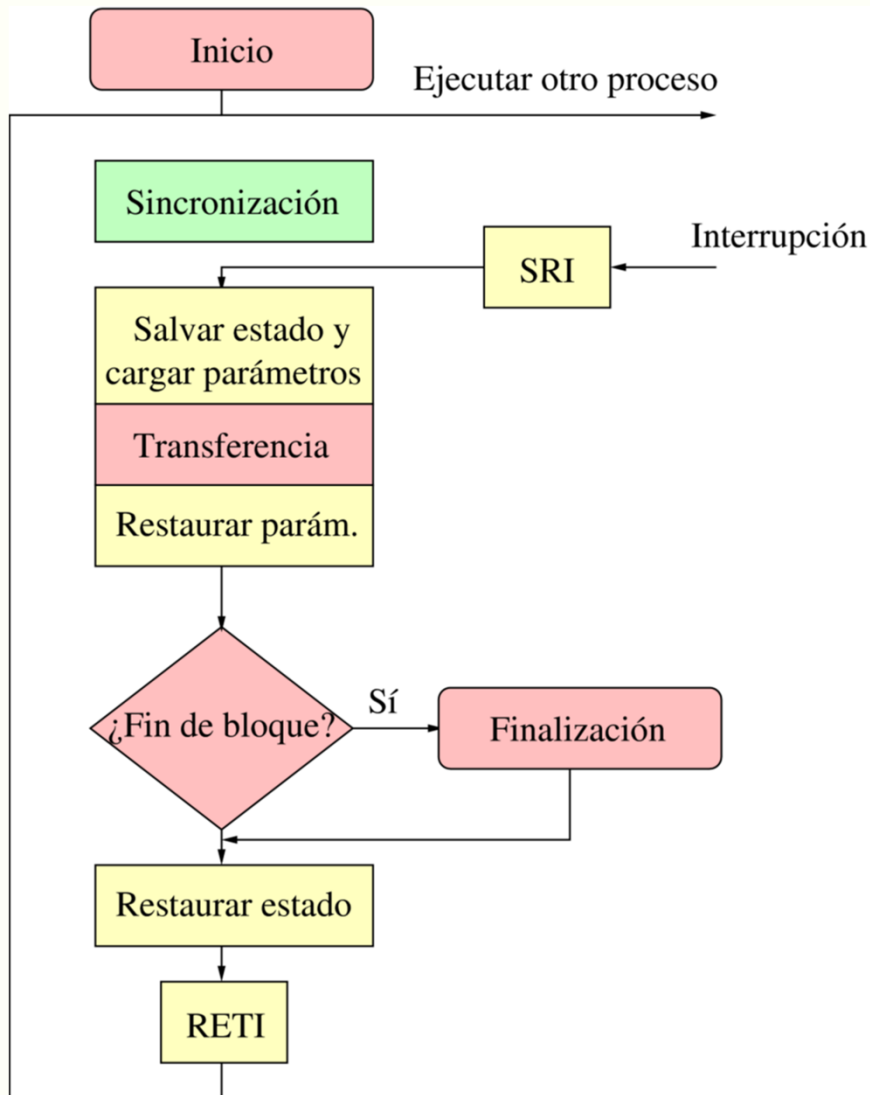


Al ejecutar RETI se restaura el nivel previo

Ejemplo de anidamiento (II)

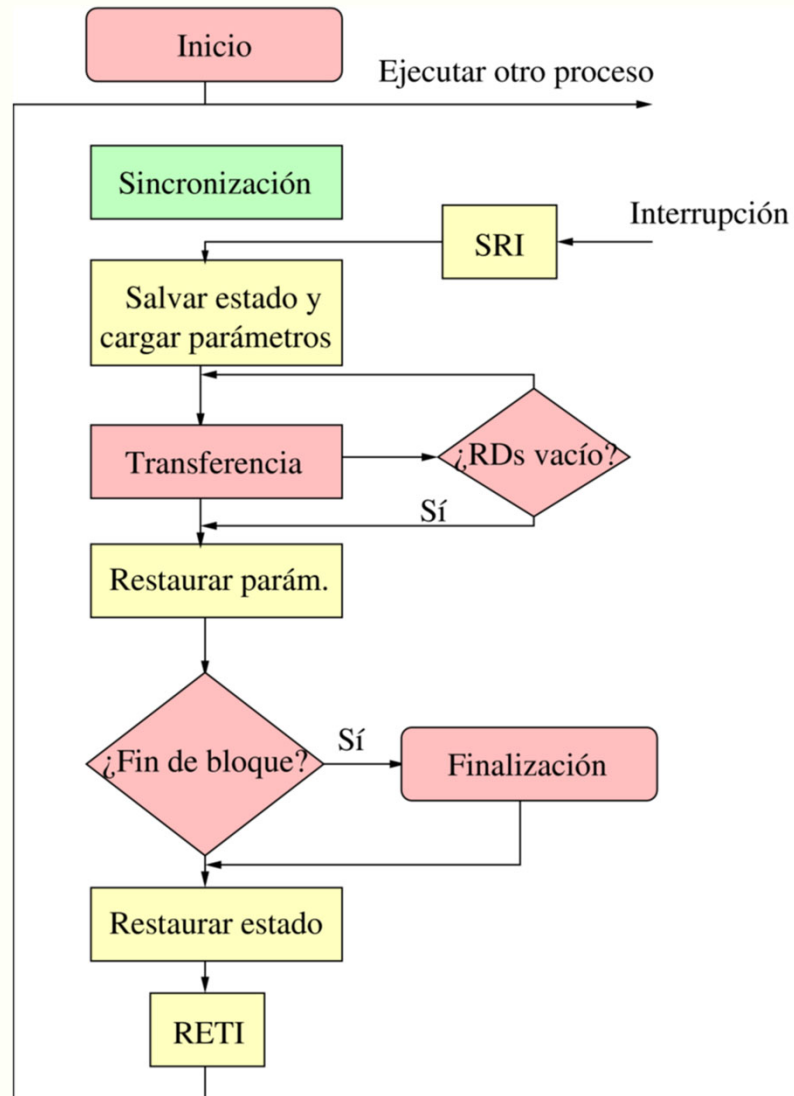


Análisis cuantitativo



- Se evita la sincronización pero se realizan otras operaciones para llevar a cabo la transferencia.
- El total supone mucho menos tiempo de CPU que por programa pero aún existe una sobrecarga inevitable.
- Para minimizar el impacto de esta sobrecarga se puede aumentar el tamaño del registro de datos.

Uso de *buffer*



- Se dota al módulo de entrada/salida de un *buffer* de registros de datos
- Solicita la interrupción cuando el *buffer* está lleno (operación de entrada) o vacío (operación de salida)
- El número de interrupciones por operación se reduce y la sobrecarga por cada dato es mucho menor en función del tamaño del *buffer*

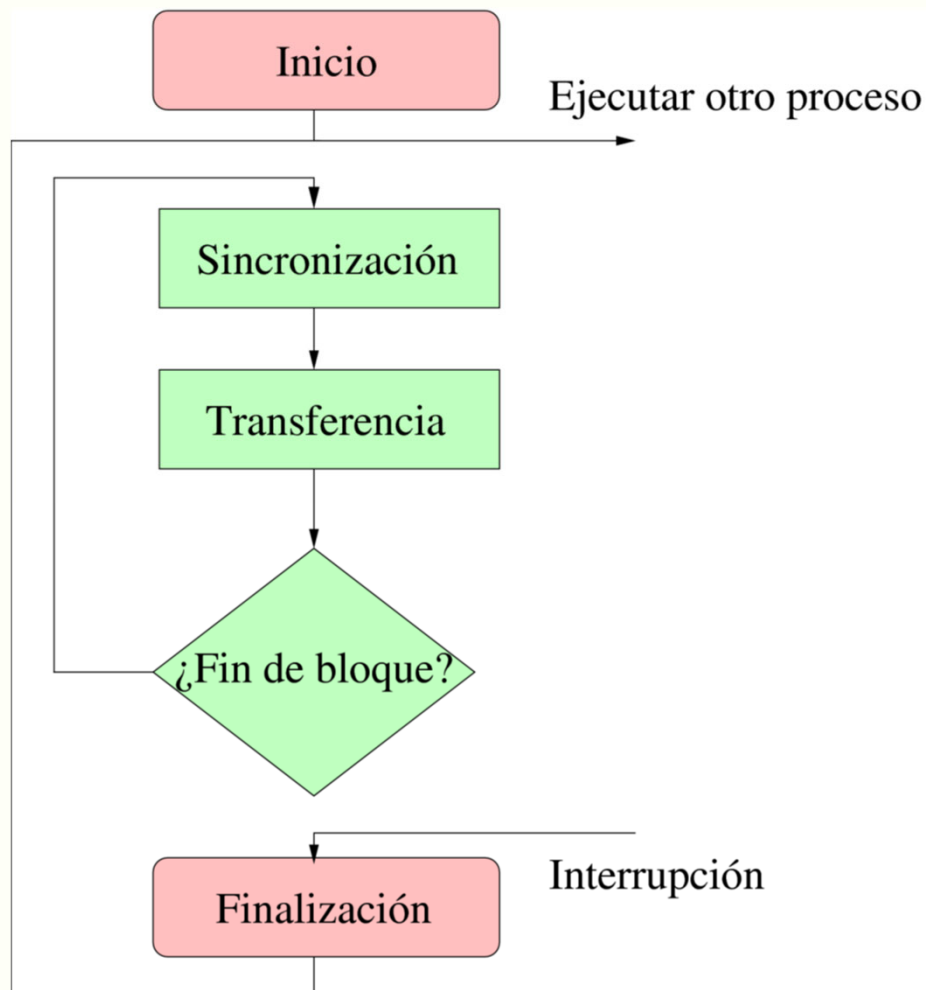
Asignación de prioridades

- Método óptimo:
asignar **mayor prioridad** al dispositivo que pide **interrupciones con mayor frecuencia**.
- La frecuencia depende de:
 - **velocidad de transferencia**
 - **tamaño del registro de datos**
- Excepciones. Algunos dispositivos, como:
 - Consola de operación
 - Temporizadores programables
 - DMA (¿prioridad de interrupción? ... más adelante)

Interrupciones no enmascarables

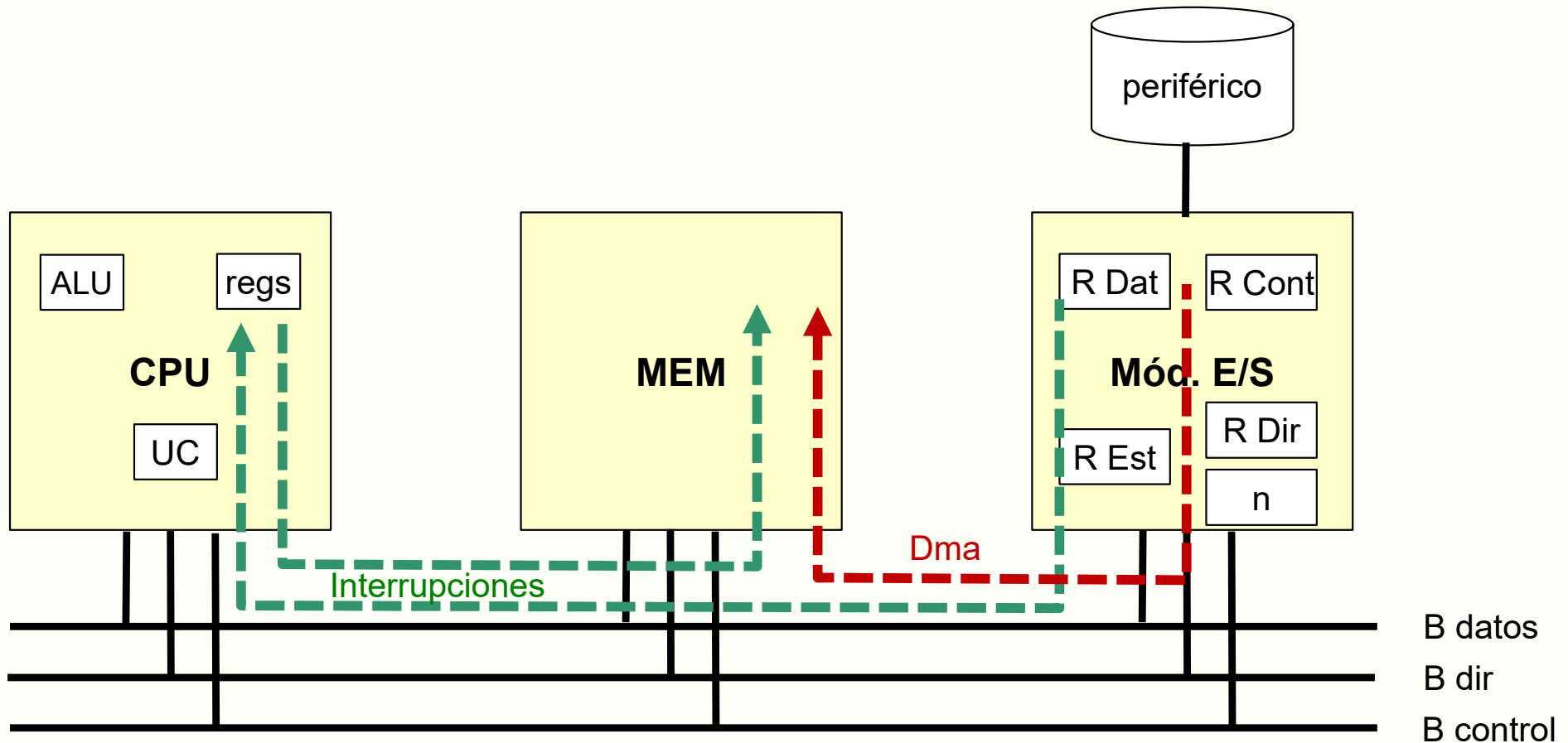
- Existen sucesos que no admiten demora en su tratamiento.
- Por ejemplo un fallo de energía.
- Para ellos existe una línea especial de petición de interrupción que **no** puede ser enmascarada:
 - NMI en la familia x86
 - INT₇ en la familia M68000

Entrada/salida mediante DMA

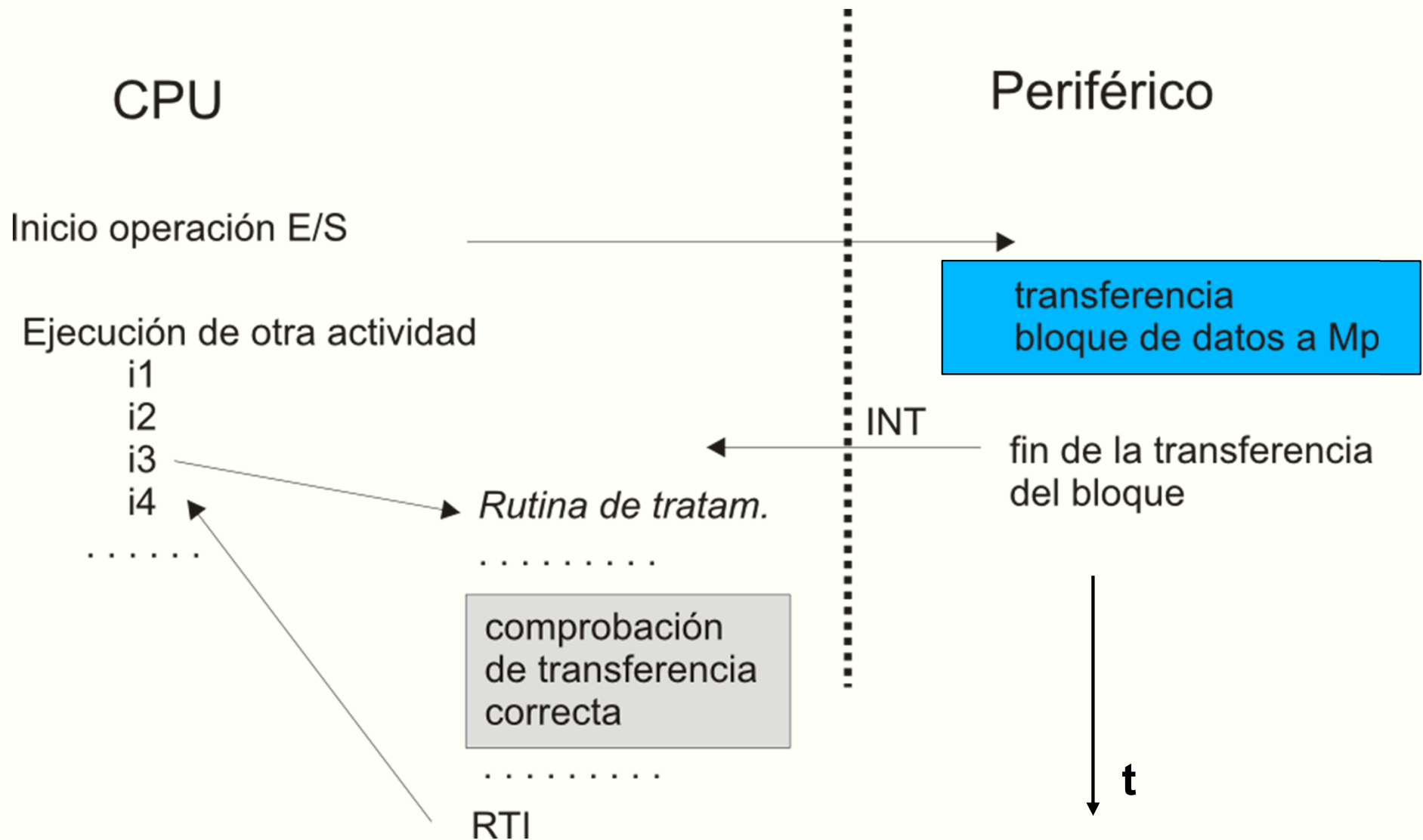


- La CPU se encarga de iniciar la operación.
- El módulo de entrada salida se encarga por Hw de la sincronización y transferencia y avisa cuando ha terminado mediante una interrupción.
- La CPU finaliza la operación.
- Hay una única interrupción por operación: se ahorra mucho tiempo de CPU con dispositivos de bloque.

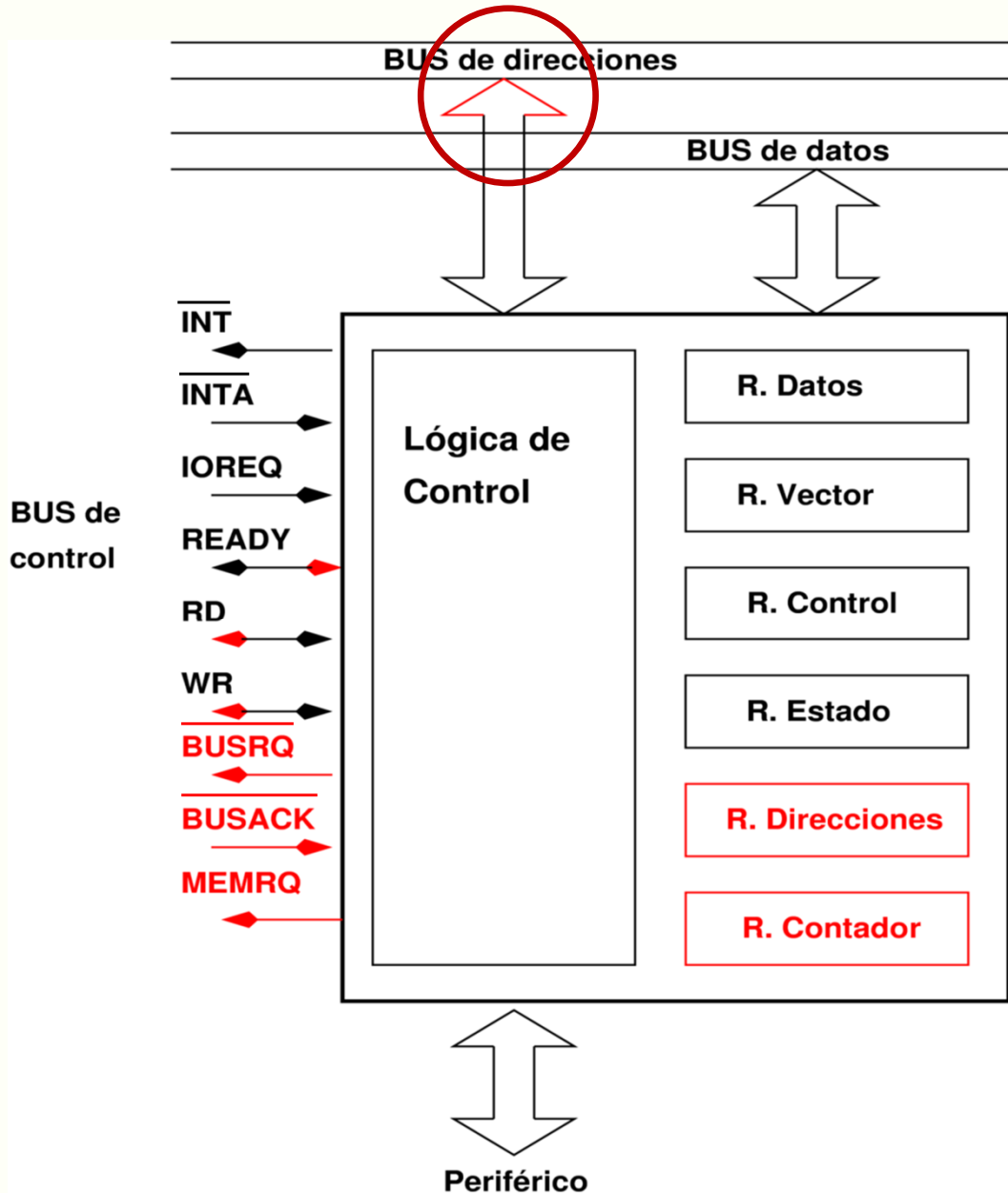
Módulo E/S con DMA



Entrada/salida mediante DMA



Módulo E/S con DMA

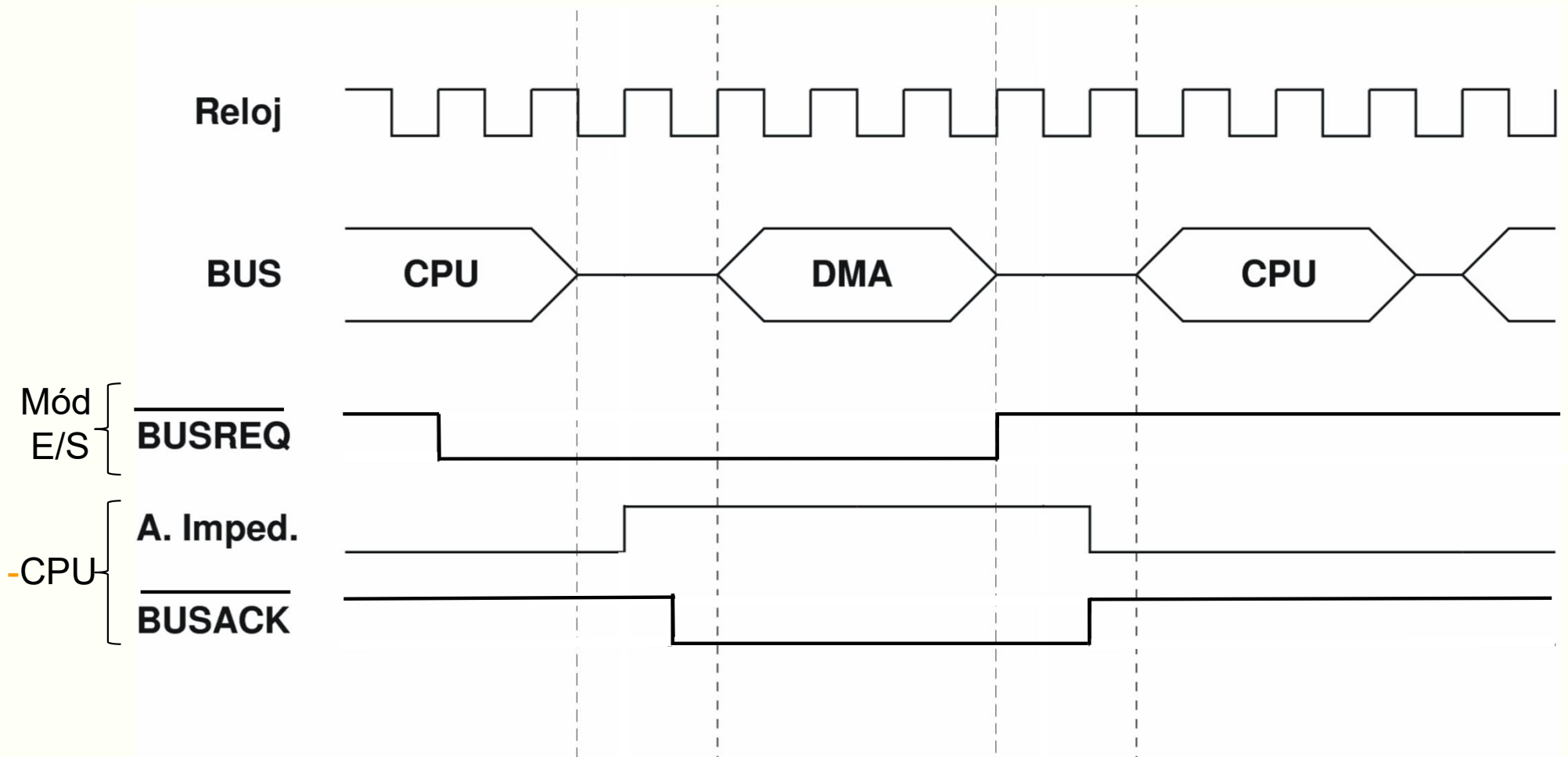


- El módulo es más complejo
- Debe generar las señales de dirección y control, para gobernar la memoria. Antes las generaba solo la UC
- Nuevos registros para el contador y Dir Memoria (antes por SW en la RTI)
- Necesita incrementador, **Dir++**, decrementador, **cont--**, y comparador

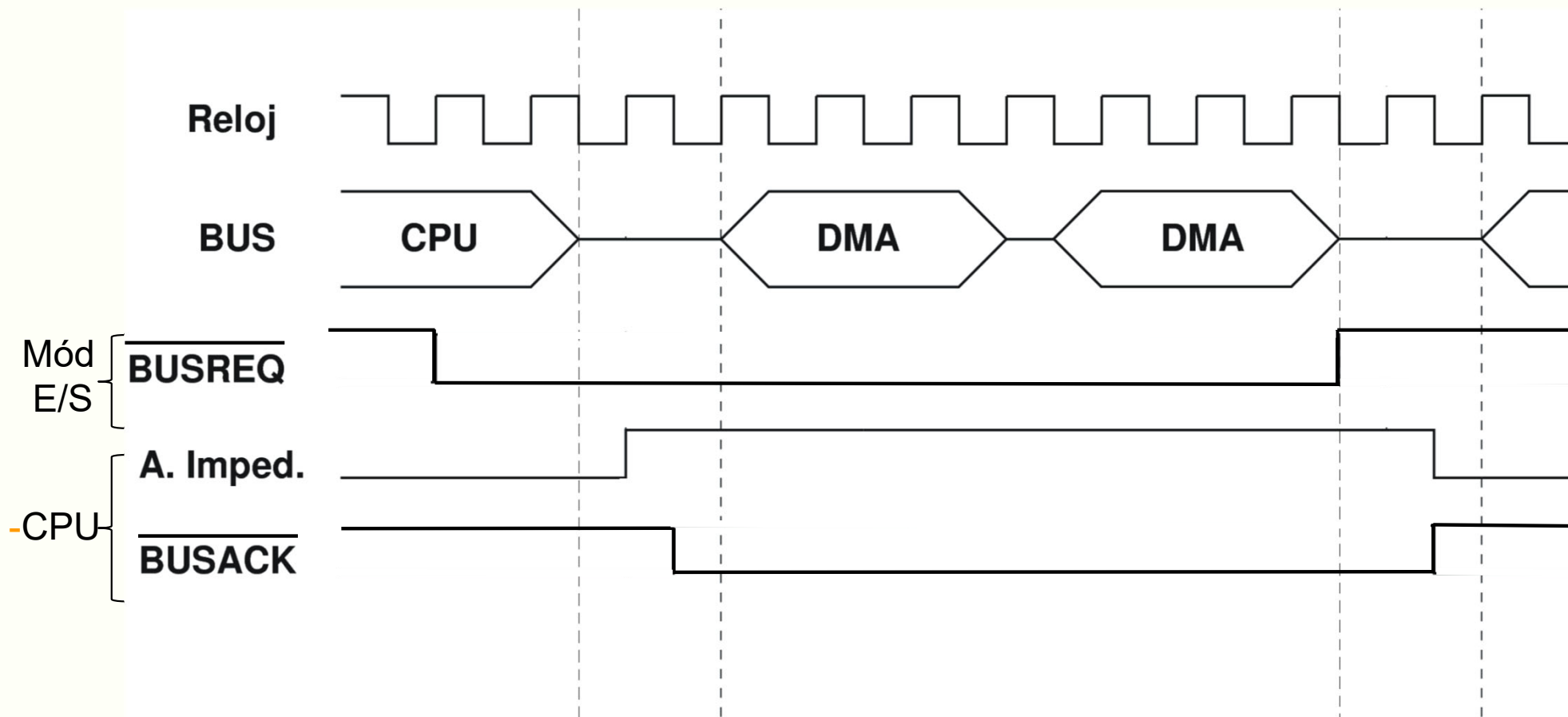
DMA mediante robo de ciclo

- El módulo, cuando hay datos listos, solicita los buses con BUSRQ.
- La CPU los cede al final del ciclo de bus en curso. Para ello, se “desconecta” de los buses colocando sus salidas en *alta impedancia*.
- La CPU indica al módulo que puede usarlos con BUSACK
- El módulo inicia el ciclo de bus para realizar la(s) transferencia(s) con memoria. Debe activar señales de dirección y control. Actualiza contador y Dir.
- Cuando acaba devuelve el bus desactivando BUSRQ.
- La CPU recupera los buses y desactiva BUSACK.
- Si no hay más datos a transferir ($\text{contador} = 0$) el módulo avisa mediante una interrupción. La CPU finaliza la E/S

Robo de ciclo aislado (sin buffer)



DMA por ráfagas (con buffer)



Operación por DMA (1)

1.- Programar la operación de DMA en el periférico: instrucciones de salida (**OUT**)

- | | | | |
|-----------------------------------|------------|---|-------------|
| 1. Dir. Comienzo zona de Memoria: | Dir. | → | DIR |
| 2. Tamaño del bloque: | Nº datos . | → | CONT |
| 3. E ó S: | E ó S | → | E/S |

- A partir de ese momento la CPU se dedicará a “sus labores”

2.- Cuando el periférico está **listo** para la transferencia de un dato, **solicita los buses**:

Activa **BUSREQ**

- La CPU “cederá” los **buses** cuando acabe la “fase” actual de la instrucción que está ejecutando:
 - “Congela” la ejecución de la instrucción (no ejecuta la siguiente Fase).
 - Pone en “alta impedancia” sus conexiones a:
 - Bus de Datos (**DB**)
 - Bus de Direcciones (**AB**)
 - Señales de control de Memoria (**RD**, **WR**, etc.)
 - Activa **BUSACK**

Operación por DMA (2)

► 3.- El periférico:

1. Transfiere el dato a/desde M activando las ss. de control y dirección pertinentes y usando el Bus de Datos.
2. $CONT \leftarrow CONT - 1$
3. $DIR \leftarrow DIR + 1$

4.- Si $CONT \neq 0$ (* faltan más datos por transferir *) entonces

Si perif. listo para transferir siguiente dato
entonces (*modo **RÁFAGA***)

- Mantiene activa **BUSREQ**
- GO TO 3

si no (***robo de ciclo** aislado*)

- Desactiva **BUSREQ**
- la CPU:
 - desactiva **BUSACK**
 - continúa ejecutando Fase siguiente

- GO TO 2

Si no (* $CONT = 0$: bloque de datos transferido *)

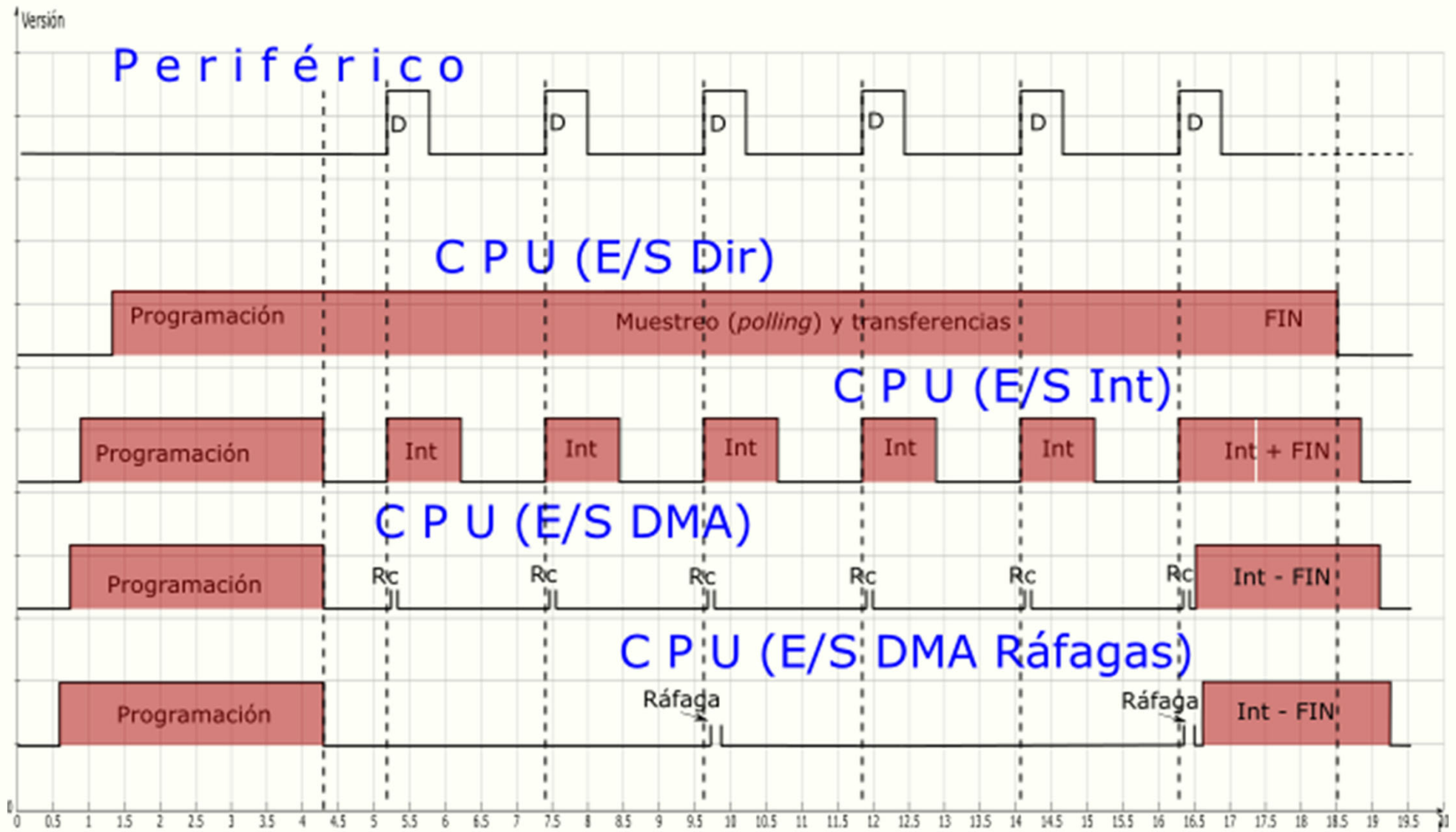
- Desactiva **BUSREQ**
- “avisa” a la CPU de que ha terminado la operación de DMA:
solicita Interrupción:

Activa **INT**

Diferencias y similitudes con las interrupciones

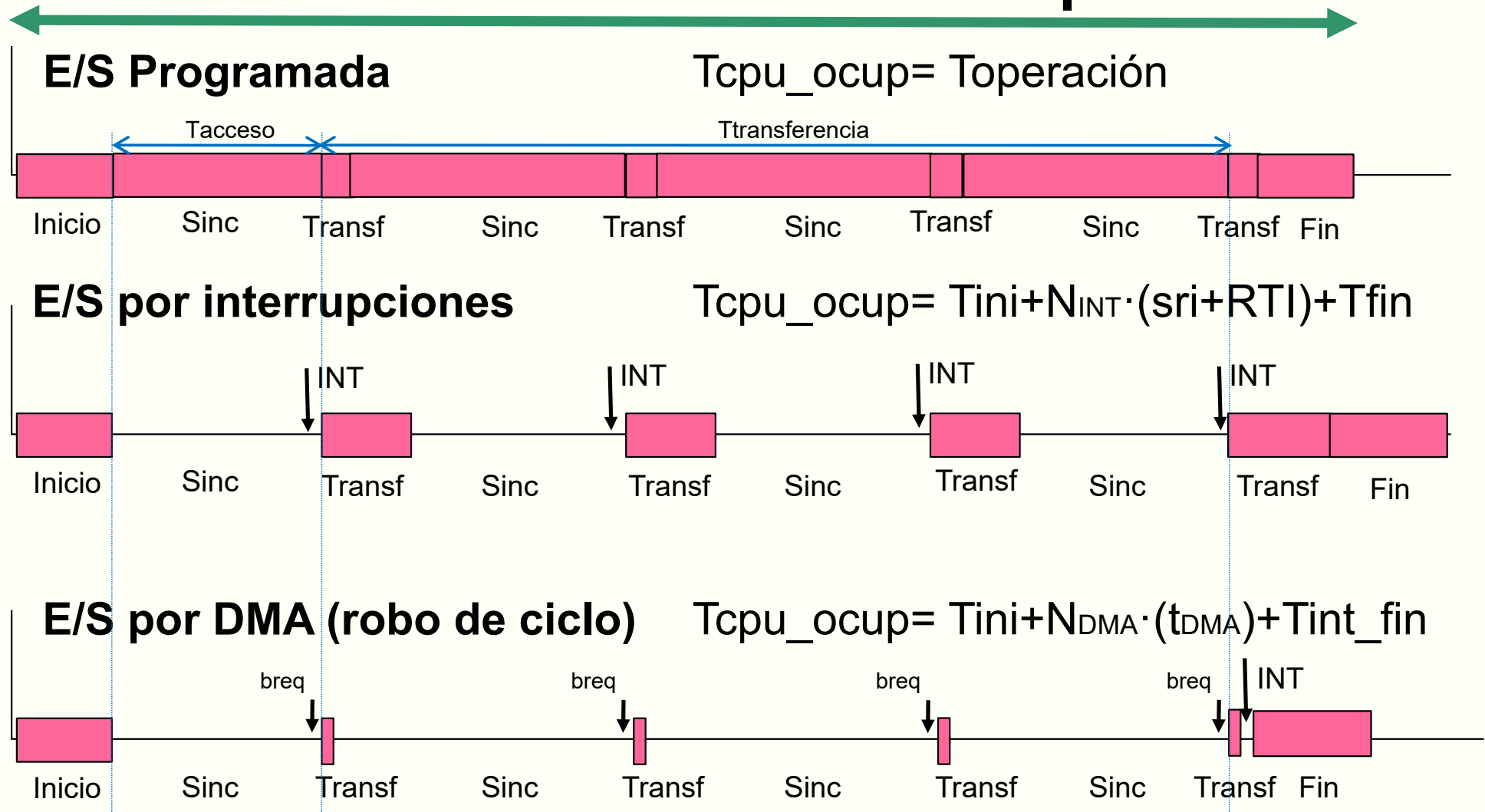
- Ambos son eventos asíncronos pero los robos de ciclo no alteran el estado de la CPU.
 - Por lo tanto se puede conceder el BUS en “cualquier” momento.
- Un ciclo de bus no es reanudable y muy breve.
 - Por lo tanto no se permite anidamiento.
- Para ambos mecanismos existe una línea para petición y otra para concesión:
 - Para peticiones de bus *simultáneas* se utilizan los esquemas de prioridad por hardware: centralizado, *daisy chain* o híbrido.

Cuadro resumen técnicas de E/S



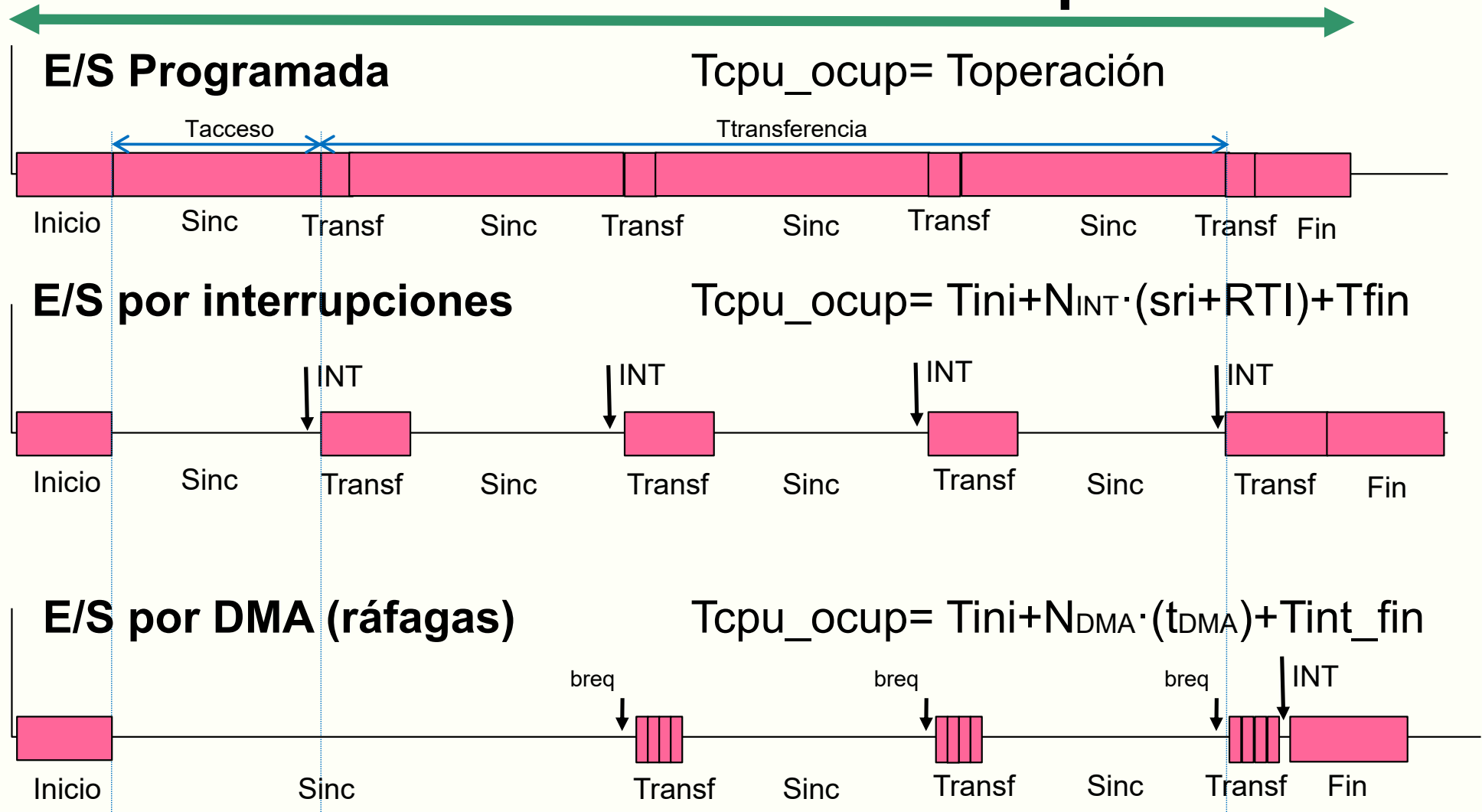
Cuadro resumen técnicas de E/S

Operación



Cuadro resumen técnicas de E/S

Tooperación



Cuadro resumen técnicas de E/S

Operación (Salida)

